YORK UNIVERSITY

Lassonde School of Engineering
Department of Electrical Engineering and Computer Science

EECS 3311: Software Design

Fall 2026: Course Project Stage 2 Report

AI Human Collaborative Implementation of FleetOps

Author:
Abhishek Saraff
220434296

Instructor: Song Wang

Date: November 10, 2026

Source code:
https://github.com/abhisheksaraff/fleet-ops

Demonstration video:
https://www.youtube.com/watch?v=1e3nmwXIGqg

Project Title: FleetOps (Heterogeneous Multi-Agent Robotics System, H-MARS)

Contents

1 Project Overview and Implementation Status **3**
1.1 What FleetOps Is . . . 3
1.2 Users . . . 3
1.3 AI and LLM Integration . . . 3
1.4 Status of Every Feature . . . 4
1.5 The Implementation in Numbers . . . 4
1.6 Limitations and Incomplete Functionality . . . 5

2 Feature to Implementation Mapping **5**
2.1 F01: Intelligence Feed Ingestion and Parsing . . . 6
2.2 F02: Deduplication of Incidents . . . 7
2.3 F03: Autonomous Task Plan Generation . . . 8
2.4 F04: Plan Approval and Manual Override . . . 9
2.5 F05: Environment and Survivor Generator, Shown on a Map . . . 11
2.6 F06: Aerial Reconnaissance by Drones . . . 12
2.7 F07: Ground Sweep with Rover and Robodogs . . . 13
2.8 F08: Helicopter Extraction . . . 14
2.9 F09: Deterministic Safety Gate . . . 15
2.10 F10: Post-Mission Analytics, Now an AI Debrief . . . 16
2.11 N01: Mission Abort and Recall (New) . . . 16

3 Implemented Design Patterns **17**
3.1 Summary and Comparison with Stage 1 . . . 18
3.2 The Patterns in Detail . . . 18
3.2.1 1. Command . . . 18
3.2.2 2. Strategy . . . 19
3.2.3 3. Composite . . . 19
3.2.4 4. Observer . . . 20
3.2.5 5. Decorator . . . 20
3.2.6 6. Facade . . . 20
3.2.7 7. Adapter . . . 21
3.2.8 8. Factory . . . 21
3.2.9 9. Template Method . . . 21
3.2.10 10. State . . . 22
3.2.11 11. Builder . . . 22
3.2.12 12. Repository . . . 22

3.2.13 13. Chain of Responsibility . . . 23
3.3 Updated UML Class Diagrams . . . 24
3.4 Updated Use-Case Diagram . . . 33
3.5 Updated Sequence Diagrams . . . 35

4 Changes and Rationale **41**
4.1 Feature Changes . . . 42
4.2 Design Pattern Changes . . . 44
4.3 Stage 1 Class to Implemented Class . . . 44
4.4 Additions Not in Stage 1 . . . 45

5 AI Tools and Collaboration Log **45**
5.1 Tools, Models, Tasks and Cost . . . 45
5.2 Gemini Usage Readings . . . 46
5.3 Division of Work . . . 46
5.4 Representative Interactions . . . 47
5.5 Keeping the AI Under Control . . . 48

6 Development and Verification Summary **48**
6.1 Weekly Commit Summary . . . 48
6.2 Checks Performed . . . 50
6.3 Defects Found and Corrected . . . 50
6.4 Remaining Limitations and Stage 3 Work . . . 51

7 Demonstration Index **51**
7.1 Feature Index . . . 51
7.2 Segment Timeline . . . 52

A Appendix: Traceability, Configuration and Usage **52**
A.1 Stage 1 Requirement to Running Demonstration . . . 53
A.2 How to Build and Run . . . 53
A.3 Command-Line Reference . . . 54
A.4 Configuration Keys . . . 54
A.5 Package Overview . . . 55

1 Project Overview and Implementation Status

1.1 What FleetOps Is

FleetOps is a Java 21 application, with a Swing window and a command-line interface, that turns disaster news into a simulated rescue. It reads news items about floods, famines, fires and earthquakes. A Gemini model decides whether each story describes a real incident and where it is. Verified incidents are stored in an H2 database. For an incident the operator picks, the model proposes how many drones, robodogs and helicopters to send and which search pattern to fly. A deterministic safety gate checks every command. The operator can edit and approve the plan, the mission then runs on a live map, and the model writes a debrief when it ends.

The robots are simulated. A ground rover carries drones and robodogs to the incident area, the drones and robodogs search it and reveal survivors, and helicopters carry the survivors out. The design rule that runs through the whole program is that the AI proposes and the rules decide: model output is validated and clamped, and nothing moves until the safety gate has accepted the command.

1.2 Users

The three user groups from Stage 1 are unchanged. Mission control operators run the scan, plan and launch steps. Researchers who want to compare LLM planning with deterministic checking can run the same incident with the AI planner or with the rule-based planner and compare the plans. Field operators who want a manual override can edit every plan before it starts and abort any mission. FleetOps is a simulation and does not control real hardware.

1.3 AI and LLM Integration

The LLM takes part in three stages of the workflow. In each stage the model's answer is untrusted input.

Stage	**What the model receives**	**What it returns, and how it is checked**	**If the AI fails**
Verify (F01)	Headline, description and, for Google News links, the text of the article itself	JSON with real/not real, confidence, place, latitude and longitude, severity, people at risk. `Assessment.fromJson` rejects wrong fields and clamps every number. One repair attempt.	The item is rejected with a reason and logged for review. Other items in the scan continue.
Plan (F03)	The incident and the live fleet state (ready drones, robodogs, helicopters, batteries)	Drones, search pattern, robodogs, helicopters and a rationale. Counts above what is ready are cut back. A rescue with people expected always gets a helicopter. One repair attempt.	`ResilientPlanner` switches to `RuleBasedPlanner` and says so in the rationale.
Debrief (F10)	A header of hard numbers (people, rescued, found, missed, minutes, units sent, operator changes) plus fenced incident text	Summary, what went well, problems and next-time advice. Lines that contradict the numbers or ask for units that do not exist are dropped.	`ResilientDebriefer` switches to a rule-written debrief.

The default model is Gemini 2.5 Flash. When its free daily allowance is used up, `FallbackLlmClient`

moves the request to the next model in `LLM_FALLBACK_MODELS` (Gemini 2.5 Flash-Lite in my configuration). `OllamaClient` adapts a local model server, and `MockLlmClient` gives a no-key, no-network mode used for offline runs. The AI therefore does more than forward a prompt: it reads full article text, reasons over live fleet state, produces structured decisions that feed a planner and a compiler, and works inside a chain of checks and fallbacks.

1.4 Status of Every Feature

All ten Stage 1 features run in the submitted application and appear in the demonstration video. None was dropped. Each one differs from the Stage 1 text in at least one way, and every difference is explained in Sections 2 and 4. One feature is new: N01, abort and recall, which Stage 1 described as a use case (UC-05) but did not list as a feature.

ID	**Feature**	**Status**	**Type (Stage 1)**	**Interface**
F01	Intelligence feed ingestion and parsing	Modified	AI (AI)	GUI, CLI
F02	Deduplication of incidents	Modified	Deterministic (Hybrid)	GUI, CLI
F03	Autonomous task plan generation	Modified	Hybrid (AI)	GUI, CLI
F04	Plan approval and manual override	Modified	Deterministic (Det.)	GUI, CLI
F05	Environment and survivor generator, shown on a map	Modified	Deterministic (Det.)	GUI
F06	Aerial reconnaissance by drones	Modified	Hybrid (Hybrid)	GUI, CLI
F07	Ground sweep with rover and robodogs	Modified	Hybrid (Hybrid)	GUI, CLI
F08	Helicopter extraction	Modified	Hybrid (Hybrid)	GUI, CLI
F09	Deterministic safety gate	Modified	Deterministic (Det.)	GUI, CLI
F10	Post-mission analytics, now an AI debrief	Modified	AI (AI)	GUI, CLI
N01	Mission abort and recall	New (UC-05)	Deterministic	GUI, CLI

1.5 The Implementation in Numbers

Java source files / lines of code	166 files, about 17,300 lines (`src/main/java/com/fleetops`)
Design patterns implemented	13 distinct patterns (Section 3)
CLI commands	6: `config`, `scan`, `incidents`, `show`, `run`, `reset`
GUI console commands	13, and the console also runs the 6 CLI commands against the same database
LLM clients	Gemini, Ollama, mock, with retry and model fallback
Code commits	30 (5 in period 1, 5 in period 2, 20 in period 3)

1.6 Limitations and Incomplete Functionality

These are the places where the program does less than Stage 1 promised, or where it can fail.

- **No automated test suite is committed.** The code was verified by scripted trial runs and throw-away check programs that are not in the repository (Section 6). Deterministic tests and the agent validation work belong to Stage 3.
- **Free-tier quota.** The Gemini free tier allowed about 20 requests per model per day when I tested. The fallback chain helps, but heavy use can still exhaust every model. The mock provider and the built-in demo news give an offline path that needs no key.
- **Stage 1 items that were not built:** vector similarity search, LLM tool calling, manual text paste for feeds, terrain obstacles and walls, survivors that drift or move fast, rover paths cleared by drones, toggling and reordering individual tasks, charts in the analytics report, and a local Gemma model (the Ollama adapter exists but I did not exercise it with Gemma). Section 4 gives the reason for each.
- **Telegram is off by default.** `TelegramChannelSource` reads public channel pages when `telegram` is listed in `NEWS_SOURCES`. I did not use it in the demonstration.
- **Simple physics.** Straight-line movement, constant speed, battery drain proportional to distance, no wind or terrain. The rover always drives to the south edge of the incident square.
- **The AI can still misjudge a story.** Rules reduce the risk (confidence of at least 0.6, a known place, at least one person at risk, no region-wide stories) but do not remove it.
- **Duplicate detection is local.** A report is a duplicate if its source URL is already stored or if it has the same type within 5 km of an active incident. The same event reported 20 km away would be treated as new.
- **The map needs internet access.** Tiles come from CARTO (free key) or OpenStreetMap. Without tiles the incidents and units are still drawn on a blank background.

2 Feature to Implementation Mapping

This section accounts for all ten Stage 1 features and the one new feature. Each table follows the mapping fields from the Stage 2 instructions. *Status* is either Implemented (as designed) or Modified (it works, but differs from Stage 1). Every difference is listed again in Section 4 in the form Original design, Implemented change, Reason, Impact. Class names in the tables are the real class names under `src/main/java/com/fleetops`, and every video link opens the demonstration at the listed time.

2.1 F01: Intelligence Feed Ingestion and Parsing

Stage 1 reference	F01; use case UC-01; sequence diagram SD01
Status or change	**Modified.** News comes from Google News, optional public Telegram channels and built-in demo stories. The AI also reads the article text, not only the headline. Manual text paste and the Telegram bot were not built.
Type	AI-based (Stage 1: AI-based). The LLM decides whether a story is a real incident and extracts place, severity and people.
How the user invokes it	GUI: the *Scan news* button, or `scan` in the console. CLI: `scan [-t flood, fire] [-v]`. No input is needed; the sources come from `NEWS_SOURCES`.
Classes and responsibilities	`ScanCommand`: CLI entry point. Opens the application context, runs the scan and prints one line per item. `MainFrame`: GUI scan button. Runs the scan on a background thread and refreshes the cards and map. `IntelligenceService`: Facade over the whole pipeline: collect, enrich, analyse, validate, store. `NewsCollector`: Picks the newest unstored item for each topic, so one scan sends at most one item per topic to the AI. `CompositeNewsSource`, `GoogleNewsRssSource`, `TelegramChannelSource`, `DemoNewsSource`: Adapters that give every feed the same `NewsSource` interface; the composite merges them in the order listed in `NEWS_SOURCES`. `GoogleArticleEnricher`, `GoogleNewsLinks`, `ArticleText`: Find the real address behind a Google link, download the page and keep the paragraphs. Any failure leaves the item with its headline. `LlmNewsAnalyzer`, `NewsPrompts`, `Assessment`: Build the prompt (article text is fenced as untrusted), parse the JSON answer and validate it. `FallbackLlmClient`, `RetryingLlmClient`, `GeminiClient`: Send the request, retry on short outages and move to the next model when the daily quota is gone.
Key methods	`IntelligenceService.scan()`, `IntelligenceService.process()`, `NewsCollector.collectOnePerTopic()`, `GoogleArticleEnricher.enrich()`, `LlmNewsAnalyzer.analyze()`, `Assessment.fromJson()`, `FallbackLlmClient.complete()`
Design patterns	Adapter, Composite (news sources), Strategy (article enricher), Facade, Decorator (retry), Chain of Responsibility (model fallback)
Collaboration, input to result	The operator starts a scan. `IntelligenceService` asks `NewsCollector` for one item per topic. The collector reads the feeds through `CompositeNewsSource` and skips articles already stored or already rejected. Each item goes through the enricher, then `LlmNewsAnalyzer` sends it through the LLM chain and converts the JSON into an `Assessment`. `process()` then applies four rules: the story is a real incident, confidence is at least 0.6, a place was found, and at least one person is at risk. An accepted item becomes an `Incident` (built with `Incident.Builder`) and is saved. The scan returns one outcome per item (created, duplicate, rejected or failed) with a reason, which the CLI prints and the GUI console shows.
Alternative and error behaviour	A failed analysis is counted; after two failures the item is rejected with the reason. An unreachable feed is listed as a source failure and the other feeds continue. Rejected items go into a review log so the AI never reads them twice. Region-wide stories and stories with nobody at risk are rejected.
Video	0:15, F01 scan from the CLI

2.2 F02: Deduplication of Incidents

Stage 1 reference	F02 Vector and Database Deduplication Tool; use case UC-02; sequence diagram SD02
Status or change	**Modified.** Duplicates are found with a database key and a distance check. The LLM tool call and the vector search were not built (Section 4, change C3).
Type	Deterministic (Stage 1: Hybrid). The AI decides whether a story is a real incident; it does not decide whether two reports match.
How the user invokes it	Automatic during every scan, in the GUI and the CLI. Duplicates appear in `incidents -a` with the state DUPLICATE.
Classes and responsibilities	`SourceUrls`: Turns an article address into a normalised key. `IncidentRepository`: Storage interface with `existsBySourceUrl`, `findBySourceUrl` and `findActive`. `JdbcIncidentRepository`: H2 implementation. The source key column is UNIQUE; a unique violation becomes `DuplicateIncidentException`. `InMemoryIncidentRepository`: Same contract without a database, used when `DB_URL=memory`. `NewsCollector`: Skips stored articles before any AI request is made, so no quota is spent on them. `IntelligenceService`: `findSameEvent()` compares a new incident with active ones of the same type within `Settings.duplicateDistanceKm` (5 km). `Incident`, `IncidentState`: `markDuplicate()` moves NEW to DUPLICATE; the state object allows it only from NEW.
Key methods	`SourceUrls.normalize()`, `IncidentRepository.existsBySourceUrl()`, `JdbcIncidentRepository.save()`, `IntelligenceService.findSameEvent()`, `Incident.markDuplicate()`
Design patterns	Repository, State, Facade
Collaboration, input to result	Duplicates are caught at three points. The collector drops an article whose key is already in the repository. After the AI has described the incident, `findSameEvent()` flags a same-type incident within 5 km as DUPLICATE and links it to the earlier one in the outcome message. Finally the database UNIQUE key rejects any article that slipped through, for example from two scans running at once.
Alternative and error behaviour	A repository failure is reported as an error for that item and nothing is stored. The same story in a second scan is skipped silently and does not use the AI.
Video	0:29, F02 second scan and incident list

2.3 F03: Autonomous Task Plan Generation

Stage 1 reference	F03; use cases UC-02 and UC-03; sequence diagram SD03
Status or change	**Modified.** The AI chooses the size and pattern of the response. A compiler builds the commands and checks them. The AI no longer writes the commands (Section 4, change C4).
Type	Hybrid (Stage 1: AI-based).
How the user invokes it	GUI: select an incident and press *Plan mission*, or `plan <id>` in the console. CLI: `run <id> --dry-run` prints the plan without launching it.
Classes and responsibilities	`MissionControl`: `plan()` checks that the incident is VERIFIED, asks the planner, compiles the plan, moves the incident to PLANNED and saves it. `PlanningContext`: Snapshot of the ready drones, robodogs and helicopters (above the launch battery level). `LlmResponsePlanner`, `PlanPrompts`: Ask the model for a `ResponseDecision`, validate it, clamp it to the ready units, allow one repair attempt, and raise helicopters to one when people are expected. `RuleBasedPlanner`: Fixed rules: more urgent incidents get more drones, one helicopter per four people. `ResilientPlanner`: Uses the rule planner when the AI planner throws, and says so in the rationale. `PlanCompiler`, `AreaGrid`: Cut the incident square into cells, share them among drones, raise the drone count until every share fits the batteries, work out the fewest drones that can do it, and build the command list. `MissionPlan`: The result: commands, notes, drone range (`minDrones`, `maxDrones`) and staging point.
Key methods	`MissionControl.plan()`, `ResponsePlanner.decide()`, `LlmResponsePlanner.decide()`, `RuleBasedPlanner.decide()`, `PlanCompiler.compile()`, `AreaGrid.split()`, `AreaGrid.assign()`
Design patterns	Strategy (planners), Decorator (`ResilientPlanner`), Facade (`MissionControl`), Command (the compiled commands)
Collaboration, input to result	`MissionControl.plan()` builds a `PlanningContext` from the live fleet and passes it to the planner. The planner returns a `ResponseDecision` with drones, pattern, robodogs, helicopters and a rationale. `PlanCompiler` turns that into routes: the rover drives to the south edge of the incident square, each drone sweeps its cells and returns, each robodog walks its circle and returns. Every command is wrapped by the safety policy (F09). The operator sees the plan with the rationale and any notes, for example that a drone count was raised so each drone can finish its share and get home.
Alternative and error behaviour	AI unavailable or answering badly twice: the rule planner decides. Area too large for the ready batteries: a `PlanningException` with the reason. Base too far from the incident: the plan is refused before launch. No rover with a ready drone: the plan is refused.
Video	0:46, F03 plan with the AI's reasoning

2.4 F04: Plan Approval and Manual Override

Stage 1 reference	F04; use case UC-03 (operator confirms deployment); sequence diagram SD04
Status or change	**Modified.** The operator edits the number of drones, robodogs and helicopters and the search pattern, not individual tasks. Every edit is compiled and checked again (Section 4, change C5).
Type	Deterministic (Stage 1: Deterministic).
How the user invokes it	GUI: the plan dialog opens after *Plan mission*; change the values, then *Launch* or *Cancel*; *Back to original plan* restores the planner's choice. Console: `edit`, `launch`, `cancel`. CLI: `run <id>` prints the plan and asks for approval; `--send-drones`, `--send-helicopters`, `--send-robodogs` and `--pattern` change it; `-y` approves without the prompt.
Classes and responsibilities	`PlanDialog`: Shows tiles, rationale, notes and the full plan. Each change calls the reviser; a problem is shown in the dialog and the last good plan stays. The drone spinner cannot go below `MissionPlan.minDrones()`. `WindowCommands`: Reads and checks the console commands `plan`, `edit`, `launch`, `cancel`. `RunCommand`: CLI options, approval prompt and the run loop. `MissionControl`: `revise()` checks the counts against the ready units and recompiles; `launch()` queues the commands and starts the incident; `cancelPlan()` returns the incident to VERIFIED. `ResponseDecision`: `edited()` returns a copy with the operator's numbers; the plan is then labelled as planned by the AI plus the operator. `CommandQueue`: Receives the verified commands at launch.
Key methods	`MissionControl.revise()`, `MissionControl.launch()`, `MissionControl.cancelPlan()`, `ResponseDecision.edited()`, `PlanDialog.review()`, `CommandQueue.enqueueAll()`
Design patterns	Command, State, Facade
Collaboration, input to result	The dialog (or the `run` prompt) holds a `MissionPlan`. An edit calls `MissionControl.revise()`, which rejects impossible counts, compiles the edited decision through the same `PlanCompiler` and the same safety policy, and returns a new plan. On launch, `MissionControl.launch()` checks that the incident is PLANNED and that the units are free, moves it to IN_PROGRESS, enqueues the commands and hands the helicopters to `RescueDispatcher`.
Alternative and error behaviour	Cancel or answering no returns the incident to VERIFIED. A plan with no drone or no helicopter cannot be launched. A request for more units than are ready is refused with the number that are ready.
Video	0:57, F04 approval in the CLI, 1:56, F04 plan dialog in the GUI



Figure 1: The F04 plan dialog (demo data, mock planner). The operator changed the drones from 3 to 5 and the pattern from lawnmower to spiral; the rationale records the change.

2.5 F05: Environment and Survivor Generator, Shown on a Map

Stage 1 reference	F05 2D Environment and Dynamic Entity Generator; use case UC-04 (monitor); sequence diagram SD05
Status or change	**Modified.** The 2D occupancy grid became a real map. There are no walls and the survivors do not move (Section 4, change C6).
Type	Deterministic (Stage 1: Deterministic).
How the user invokes it	GUI: incidents appear on the map after a scan. Zoom and pan the map, click a marker or a card, pick a layout preset, set the simulation speed. There is no CLI form; the CLI prints the same state as text.
Classes and responsibilities	`MapPanel`, `MapTiles`: JXMapViewer2 map with CARTO dark or OpenStreetMap tiles, selection and zoom. `MapOverlay`, `Sprites`, `Icons`: Draw incident squares, mission routes, found (red) and rescued (green) survivors and the moving units as pixel-art sprites. Hidden survivors are never drawn. `World`, `IncidentZone`: The simulated world: fleet, one zone per launched incident, survivors. `SurvivorSpawner`: Places 3 to 20 hidden survivors in a zone from a seeded random generator, so a run can be repeated. `SimulationEngine`: Advances time in fixed one-second ticks and publishes events. `MainFrame`: Registers as a listener and repaints the map from the events.
Key methods	`SurvivorSpawner.spawn()`, `SimulationEngine.addIncident()`, `SimulationEngine.tick()`, `World.zoneFor()`, `MapPanel.fitAll()`
Design patterns	Observer, Factory (`SurvivorSpawner`, `UnitFactory`)
Collaboration, input to result	On launch, `SimulationEngine.addIncident()` asks `SurvivorSpawner` for the survivors and adds an `IncidentZone` to the `World`. Every tick, the queued commands move units through the engine, and the engine publishes `UnitMoved`, `UnitStateChanged`, `SurvivorFound` and `SurvivorRescued` events. `MainFrame` receives them as a `SimulationListener` and repaints; the engine itself contains no GUI code.
Alternative and error behaviour	No tile access: the map background stays blank while incidents and units are still drawn. No CARTO key: the dark style falls back to OpenStreetMap.
Video	2:43, F05 map, incident square and units

2.6 F06: Aerial Reconnaissance by Drones

Stage 1 reference	F06; use case UC-06; sequence diagram SD06
Status or change	**Modified.** Drones sweep their share of the incident square and reveal survivors. They do not clear fog of war or paths for the rover (Section 4, change C7).
Type	Hybrid (Stage 1: Hybrid). The AI picks the number of drones and the pattern; the paths, battery checks and detection are deterministic.
How the user invokes it	Part of every mission. The operator sets the number of drones and the pattern (lawnmower or spiral) in the plan dialog, with `edit`, or with `--send-drones` and `--pattern`.
Classes and responsibilities	`Drone`, `Sensing`: A unit with a 0.25 km sensor radius and a battery drain per kilometre. `SweepAreaCommand`: Flies the path from a `SearchStrategy` over one cell and reveals survivors inside the sensor radius. `LawnmowerStrategy`, `SpiralStrategy`: Two interchangeable ways to cover a square. `AreaGrid`: Splits the square into cells and shares them among drones. `AwaitCommand`: Holds a drone until the rover has reached the staging point. `ReturnToBaseCommand`: Brings the drone back to the rover. `SimulationEngine`: `revealSurvivors()` marks survivors found within range and publishes `SurvivorFound`.
Key methods	`SweepAreaCommand.onStep()`, `SearchStrategy.computePath()`, `AreaGrid.assign()`, `SimulationEngine.revealSurvivors()`, `AbstractMissionCommand.step()`
Design patterns	Command, Strategy, Template Method, Decorator
Collaboration, input to result	The compiled plan gives each drone a lane: wait for the rover, sweep its cells, return. `CommandQueue` runs lanes in parallel, one command per unit per tick. `SweepAreaCommand` follows the path computed by its strategy, calls the engine to reveal survivors near each position, and finishes when the path ends. Each found survivor produces an event that F08 reacts to.
Alternative and error behaviour	If the batteries cannot cover the area the compiler raises the drone count; if no count works the plan is refused. At run time a command that fails the safety rules is rejected and the rest of that drone's lane is cancelled. The 15% reserve is checked before each command.
Video	2:53, F06 drones sweeping and finding survivors

2.7 F07: Ground Sweep with Rover and Robodogs

Stage 1 reference	F07; use case UC-07; sequence diagram SD07
Status or change	**Modified.** The rover drives to the south edge of the square and drops robodogs, which walk circles. There is no cleared path and no blocked-path replanning (Section 4, change C8).
Type	Hybrid (Stage 1: Hybrid). The AI chooses how many robodogs; the circle radius and routes come from rules.
How the user invokes it	Part of a mission when robodogs are above zero. GUI: the *Robodogs to drop* spinner, or `edit robodogs=N`. CLI: `--send-robodogs N`.
Classes and responsibilities	`Carrier`: The rover. It is a unit and also the composite that carries drones and robodogs; `hold()` and `recall()` reach everything on board. `Robodog`, `Sensing`: A unit with a 0.15 km sensor radius. `FlyToCommand`: Moves the rover to the staging point and each robodog to its drop-off point. `SweepAreaCommand`, `RadiusSweepStrategy`: Walks circles outward from the drop-off point. `PlanCompiler`: Sets each robodog's circle (radius up to 0.4 km) and skips a robodog whose battery cannot cover the walk, the circle and the way back. `UnitFactory`: Creates robodogs with unique ids.
Key methods	`Carrier.addRobodog()`, `Carrier.availableRobodogs()`, `RadiusSweepStrategy.computePath()`, `SweepAreaCommand.onStep()`, `UnitFactory.createRobodog()`
Design patterns	Composite, Strategy, Command, Factory
Collaboration, input to result	The rover is a drive command followed by nothing; its robodogs wait for it, walk to their drop-off points along the south edge, sweep a circle with the radius strategy and return. Survivors they reveal produce the same events as those found by drones, so F08 treats both the same way.
Alternative and error behaviour	A robodog that cannot make the trip stays on the rover and the plan notes say why. If the operator asks for more robodogs than are ready, the request is refused with the number available.
Video	3:06, F07 robodogs dropped by the rover

2.8 F08: Helicopter Extraction

Stage 1 reference	F08; use case UC-08; sequence diagram SD08
Status or change	**Modified.** Dispatch is automatic only. There are no landing-zone obstacles and no manual dispatch by clicking a survivor (Section 4, change C9).
Type	Hybrid (Stage 1: Deterministic and hybrid). The AI sets how many helicopters are ready; dispatch is deterministic.
How the user invokes it	Automatic after a survivor is found. The operator sets the number of helicopters in the plan (`--send-helicopters`, the *Helicopters ready* spinner or `edit helicopters=N`).
Classes and responsibilities	`RescueDispatcher`: Listens for `SurvivorFound`, keeps a waiting list per incident and, on every tick, matches the nearest free helicopter with room that the safety policy accepts. `Helicopter`: A unit with a capacity; `board()` and `unloadAll()`. `PickUpCommand`: Flies to a found survivor and picks them up. It fails at once if the survivor was not found, is already rescued, or the helicopter is full. `ReturnToBaseCommand`: Takes the helicopter and its passengers home. `SimulationEngine`: `rescue()` marks the survivor rescued and publishes `SurvivorRescued`.
Key methods	`RescueDispatcher.onEvent()`, `RescueDispatcher.dispatch()`, `PickUpCommand.onStep()`, `Helicopter.board()`, `Helicopter.unloadAll()`, `SimulationEngine.rescue()`
Design patterns	Observer, Command, Decorator (pick-ups are verified like every command)
Collaboration, input to result	A drone or robodog finds a survivor and the engine publishes the event. `RescueDispatcher` adds the survivor to its waiting list. On the next tick it asks the safety policy about the nearest helicopter, using the helicopter's real position and battery, and if the answer is yes it enqueues a verified `PickUpCommand`. The survivor is marked rescued, the green dot appears, and the helicopter flies home and unloads. The mission ends when everyone is rescued or no more can be.
Alternative and error behaviour	If no helicopter is free or a pick-up would strand one, the survivor stays on the waiting list. People found but not rescued are counted separately in the mission report. An abort stops new pick-ups.
Video	3:23, F08 helicopter pick-up

2.9 F09: Deterministic Safety Gate

Stage 1 reference	F09 Deterministic ActionVerifier Safety Gate; use case UC-03; sequence diagram SD09
Status or change	**Modified.** Three rules (unit ready, battery reserve, service area) replace the wall, water and map-bound checks, because there is no terrain. The gate is applied at plan time and again at run time (Section 4, change C10).
Type	Deterministic (Stage 1: Deterministic).
How the user invokes it	Automatic, with no screen of its own. The operator sees it as a refused plan, a note in the plan, or a command shown as rejected in the CLI progress and the GUI console.
Classes and responsibilities	`SafetyPolicy`: Holds the rule list. `standard(base)` builds the default policy. `evaluate()` returns a `SafetyReport`; `verify()` wraps a command. `SafetyRule`: One rule: `check(command, start)` returns a message when the command is unsafe. `UnitReadyRule`: The unit must be idle and not depleted. `BatteryReserveRule`: The battery must cover the job, the trip home and a 15% reserve. `ServiceAreaRule`: The command must stay within 100 km of the base. `VerifiedCommand`: Decorator that checks the rules on its first step, with the unit's real position and battery, and fails the command with the rule name and reason if any rule is broken.
Key methods	`SafetyPolicy.evaluate()`, `SafetyPolicy.verify()`, `SafetyRule.check()`, `BatteryReserveRule.check()`, `VerifiedCommand.step()`
Design patterns	Decorator, Strategy
Collaboration, input to result	`PlanCompiler` evaluates the rover's drive with the policy and refuses the plan if it is unsafe, then wraps every other command with `policy.verify()`. At run time, the first step of a `VerifiedCommand` asks the policy again with the unit's current state. A failed check marks the command FAILED with a message such as "needs 31.2% battery (job 20.5% + trip home 5.7% + reserve 15.0%) but has 29.0%" and cancels the rest of that unit's lane. Other units carry on. `RescueDispatcher` asks the same policy before it queues a pick-up.
Alternative and error behaviour	The AI cannot bypass the gate: every command, whether from the AI planner, the rule planner, the operator's edit or the dispatcher, passes through the same policy. There is no automatic AI replan after a rejection; the operator sees the reason and decides.
Video	1:24, F09 a command refused by the safety gate

2.10 F10: Post-Mission Analytics, Now an AI Debrief

Stage 1 reference	F10 Post-Mission Ground-Truth Analytics and Reporting; use case UC10; sequence diagram SD10
Status or change	**Modified.** The report is a short debrief (summary, what went well, problems, what to change next time) built from the mission's hard numbers. There are no charts (Section 4, change C11).
Type	AI-based with a rule-based fallback (Stage 1: AI-based).
How the user invokes it	Automatic when a mission ends. GUI: the mission card shows the debrief, and `debrief [id]` in the console prints it. CLI: `run` prints the report and then the debrief unless `--no-debrief` is given.
Classes and responsibilities	`MissionReport`: Outcome of the mission: people, rescued, found but not rescued, never found, simulated time, units sent and any operator change to the plan. `DebriefFacts`, `DebriefPrompts`: Write the numbers as a header for the model and give it the rules for what it may say. `LlmDebriefer`: Asks the model, validates the JSON field by field, drops empty or contradicting claims. `PlainDebriefer`, `DebriefRules`: Write a debrief from rules when the AI fails; `advicePossible()` keeps advice within the units that exist. `AbstractDebriefer`: Template Method: `debrief()` is fixed, subclasses implement `write()`. `ResilientDebriefer`: Tries the AI debriefer first and falls back to rules. `Debrief`: The result record; text is tidied and each list is cut to a few short items. `MissionCard`, `RunCommand`: Show the debrief in the GUI and the CLI.
Key methods	`MissionListener.onMissionFinished()`, `MissionDebriefer.debrief()`, `AbstractDebriefer.debrief()`, `LlmDebriefer.write()`, `DebriefFacts.toHeader()`, `DebriefRules.advicePossible()`
Design patterns	Template Method, Strategy, Decorator, Observer
Collaboration, input to result	When the last unit is home, `MissionControl` builds a `MissionReport` and calls the mission listeners. The GUI (or `RunCommand`) passes the report to the debriefer on a background thread. `LlmDebriefer` writes the facts header, sends it with the prompt rules through the LLM chain and reads back JSON. The answer must have at least one next-time item, may not claim anything about a zero count, and may not advise more units than were available; otherwise there is one repair attempt and then `PlainDebriefer` writes the debrief.
Alternative and error behaviour	AI failure of any kind gives the rule-written debrief, and the author line says so. An aborted mission gets a debrief that reports the abort instead of blaming the plan.
Video	1:38, F10 debrief of an aborted mission (CLI), 3:49, F10 debrief on the mission card (GUI)

2.11 N01: Mission Abort and Recall (New)

Stage 1 reference	New as a feature. Stage 1 described it as use case UC-05, Override or Abort Mission, but gave it no feature ID.
Status or change	**New.** It stops a mission and brings the units home.
Type	Deterministic.
How the user invokes it	GUI: *Abort* on the mission card or sidebar, *Abort all*. Console: `abort [id|all]`. CLI: `run <id> --abort-after SECONDS` stops the mission after that many simulated seconds.

Classes and responsibilities	`MissionControl`: `abort()` and `abortAll()` stop the dispatcher, cancel the mission's queued commands and send the units home. `Mission`: Tracks the phase (searching, returning, finished) and the abort request. `CommandQueue`: `cancelWhere()` cancels pending and running commands that match a condition. `ReturnToBaseCommand`: Brings each drone and robodog back to the rover. `SimulationEngine`: `hold()` stops the rover and everything it carries. `RescueDispatcher`: `stop()` ends new pick-ups for that incident.
Key methods	`MissionControl.abort()`, `MissionControl.abortAll()`, `CommandQueue.cancelWhere()`, `RescueDispatcher.stop()`, `SimulationEngine.hold()`
Design patterns	Command, Observer, Facade
Collaboration, input to result	`MissionControl.abort()` marks the mission, stops the dispatcher, cancels every queued command for the mission's units, holds the rover and queues a return-to-base command for each drone or robodog that is still out. The mission ends with the status ABORTED and still gets a report and a debrief.
Alternative and error behaviour	If the mission is already on its way home, abort only stops new pick-ups so that units already flying are not interrupted. Aborting a finished mission does nothing.
Video	1:38, N01 abort from the CLI

3 Implemented Design Patterns

The application uses 13 distinct design patterns. Each one solves a specific problem in the code; none was added only to raise the count. Section 3.1 summarises all of them against Stage 1, Section 3.2 describes each one, and Sections 3.3 to 3.5 give the updated UML class diagrams, use-case diagram and sequence diagrams.

#	Pattern	Where it is used (main classes)	Stage 1

3.1 Summary and Comparison with Stage 1

#	Pattern	Where it is used (main classes)	Stage 1
1	Command	`MissionCommand`, `AbstractMissionCommand`, `FlyToCommand`, `SweepAreaCommand`, `PickUpCommand`, `ReturnToBaseCommand`, `AwaitCommand`, `CommandQueue`	Matches (`Command`, `ScanAreaCommand`)
2	Strategy	`SearchStrategy`, `ResponsePlanner`, `MissionDebriefer`, `SafetyRule`, `ArticleEnricher`	Matches, and used in more places
3	Composite	`FleetComponent`, `Fleet`, `Carrier`, `Unit`; `CompositeNewsSource`	Matches (`Robot`, `TrackedRover`)
4	Observer	`SimulationEngine`, `SimulationListener`, `MissionListener`, `CommandListener`	Matches (`GridObserver`)
5	Decorator	`VerifiedCommand`, `RetryingLlmClient`, `ResilientPlanner`, `ResilientDebriefer`	Changed (`ActionVerifier` became a true decorator)
6	Facade	`IntelligenceService`, `MissionControl`, `AppContext`	Matches (`PlanOrchestrator` became `MissionControl`)
7	Adapter	`GeminiClient`, `OllamaClient`, `GoogleNewsRssSource`, `TelegramChannelSource`	Matches (`TelegramAdapter`, `GeminiClient`)
8	Factory	`UnitFactory`, `LlmClients`, `SurvivorSpawner`	Changed (`TaskFactory` removed)
9	Template Method	`AbstractMissionCommand`, `AbstractAppCommand`, `AbstractDebriefer`	Listed for F10 only; now three uses
10	State	`IncidentState`, `UnitState`	New
11	Builder	`Incident.Builder`	New
12	Repository	`IncidentRepository`, `JdbcIncidentRepository`, `InMemoryIncidentRepository`	New (replaces `DatabaseTool`)
13	Chain of Responsibility	`FallbackLlmClient`	New

3.2 The Patterns in Detail

3.2.1 1. Command

Problem	The AI planner, the operator and the dispatcher all request actions from different kinds of robots. Those actions take many ticks, may fail, may be cancelled, and must be inspected by the safety gate before they run.
Participants and roles	`MissionCommand`: the command interface. `AbstractMissionCommand` and its subclasses `FlyToCommand`, `SweepAreaCommand`, `PickUpCommand`, `ReturnToBaseCommand`, `AwaitCommand`: concrete commands. `CommandQueue`: the invoker, with one lane per unit. `Unit` through `SimulationEngine`: the receivers.

Files and methods	`command/*.java`. `MissionCommand.step()`, `cancel()`, `requiredBatteryPercent()`, `endPosition()`; `CommandQueue.enqueueAll()`, `cancelWhere()`, `onTick()`.
How it follows the pattern	A request is an object. `CommandQueue` runs the first command of each unit's lane on every tick, runs different units in parallel, cancels the rest of a lane when a command fails, and cancels by condition on abort. The planner builds a plan as a list of command objects, so the operator can read it and the safety gate can wrap it before anything runs.
Stage 1 match	Matches. `execute()` became `step(engine)` because a command runs over many ticks, and `getRequiredEnergy()` became `requiredBatteryPercent(start)`. `ScanAreaCommand`, `TraversePathCommand` and `EvacuateSurvivorCommand` became `SweepAreaCommand`, `FlyToCommand` and `PickUpCommand`.

3.2.2 2. Strategy

Problem	Several algorithms do the same job and must be swappable at run time: how to cover an area, who decides the size of a response, who writes a debrief, which safety rules apply, how much of an article the AI reads.
Participants and roles	`SearchStrategy` with `LawnmowerStrategy`, `SpiralStrategy` and `RadiusSweepStrategy`. `ResponsePlanner` with `LlmResponsePlanner` and `RuleBasedPlanner`. `MissionDebriefer` with `LlmDebriefer` and `PlainDebriefer`. `SafetyRule` with three rules. `ArticleEnricher` with `GoogleArticleEnricher` and `NONE`.
Files and methods	`strategy/*.java`, `plan/ResponsePlanner.java`, `debrief/MissionDebriefer.java`, `safety/SafetyRule.java`, `news/ArticleEnricher.java`. `SearchStrategy.computePath()`, `ResponsePlanner.decide()`, `MissionDebriefer.debrief()`, `SafetyRule.check()`, `ArticleEnricher.enrich()`.
How it follows the pattern	The client holds the interface and never the concrete class. `SweepAreaCommand` takes any `SearchStrategy`, so the same command serves drones (lawnmower or spiral) and robodogs (radius sweep). The operator picks the drone pattern in the plan dialog. `AppContext` picks the planner and debriefer from configuration.
Stage 1 match	Matches for search patterns (`SearchStrategy`, `LawnmowerScanStrategy`). Extended to `SpiralStrategy`, `RadiusSweepStrategy` and four other families.

3.2.3 3. Composite

Problem	A fleet, a rover with its drones and robodogs, and a single drone should all answer the same questions: where are the units, which are ready, recall, hold. News feeds have the same shape: several feeds should behave like one.
Participants and roles	`FleetComponent`: component interface. `Unit` (with `Drone`, `Robodog`, `Helicopter`): leaf. `Fleet`: composite of components. `Carrier`: a unit that is also a composite of its drones and robodogs. `NewsSource` and `CompositeNewsSource`: the same idea for feeds.
Files and methods	`domain/fleet/*.java`, `news/CompositeNewsSource.java`. `units()`, `recall()`, `hold()`, `availableUnits()`, `findUnit()`, `CompositeNewsSource.fetch()`.
How it follows the pattern	`Carrier.units()` returns the rover plus every drone and robodog on it, so `engine.hold(carrier)` during an abort stops the whole group with one call. `CompositeNewsSource` merges feeds in the order of `NEWS_SOURCES`, so real news is read before demo stories.
Stage 1 match	Matches. `Robot`, `TrackedRover` and `Robodog` became `FleetComponent`/`Unit`, `Carrier` and `Robodog`. The news composite is an addition.

3.2.4 4. Observer

Problem	The simulation must not know about the window, the console or the dispatcher, but all of them need to react when a unit moves, a survivor is found or a mission ends.
Participants and roles	`SimulationEngine`: subject. `SimulationListener`: observer interface, receiving sealed `SimulationEvent` records. `RescueDispatcher` and `MainFrame`: observers. `MissionControl` with `MissionListener`, and `CommandQueue` with `CommandListener`: two more subject and observer pairs.
Files and methods	`sim/SimulationEngine.java`, `sim/SimulationListener.java`, `sim/SimulationEvent.java`, `mission/MissionListener.java`, `command/CommandListener.java`. `addListener()`, `publish()`, `onEvent()`, `onMissionFinished()`, `onCommandFinished()`.
How it follows the pattern	Every change to the world goes through an engine method that publishes an event, so no state change can be missed by a listener. `MainFrame` registers `rig.engine().addListener(this::onEvent)`, and the engine imports no GUI class.
Stage 1 match	Matches in purpose. `GridObserver`, `GridEngine` and `GridCanvasGUI` became `SimulationListener`, `SimulationEngine` and `MainFrame` with `MapOverlay`, because the grid became a map.

3.2.5 5. Decorator

Problem	Add a behaviour around an object without changing the object: a safety check before a command, retries around an AI call, a rule-based fallback around an AI planner or debriefer.
Participants and roles	`VerifiedCommand` wraps a `MissionCommand`. `RetryingLlmClient` wraps an `LlmClient`. `ResilientPlanner` wraps a `ResponsePlanner` pair (preferred and fallback). `ResilientDebriefer` does the same for `MissionDebriefer`.
Files and methods	`safety/VerifiedCommand.java`, `ai/RetryingLlmClient.java`, `plan/ResilientPlanner.java`, `debrief/ResilientDebriefer.java`. `VerifiedCommand.step()`, `RetryingLlmClient.complete()`, `ResilientPlanner.decide()`, `ResilientDebriefer.debrief()`.
How it follows the pattern	Each wrapper implements the same interface as the object it wraps and delegates to it. `VerifiedCommand` implements `MissionCommand`, so `CommandQueue` cannot tell a verified command from a plain one and cannot skip the check.
Stage 1 match	Changed. Stage 1 had a standalone `ActionVerifier` class described as a decorator and safety gate. It is now a real decorator (`VerifiedCommand`) that gets its rules from a `SafetyPolicy`. The retry and fallback wrappers are new.

3.2.6 6. Facade

Problem	The GUI and the CLI need simple entry points over many collaborating classes.
Participants and roles	`IntelligenceService`: scan, enrich, analyse, deduplicate and store. `MissionControl`: plan, revise, launch, abort and run missions. `AppContext`: builds and shares configuration, LLM client, repository, news source, planner and debriefer.
Files and methods	`intel/IntelligenceService.java`, `mission/MissionControl.java`, `app/AppContext.java`. `scan()`, `plan()`, `revise()`, `launch()`, `abort()`, `runUntilSettled()`, `llm()`, `repository()`, `planner()`.
How it follows the pattern	The GUI and each CLI command call these three classes only. For example, `RunCommand` never touches the planner, compiler or queue; it calls `MissionControl`.

Stage 1 match	Matches. `IntelligenceService` kept its name. `PlanOrchestrator` became `MissionControl` together with `ResponsePlanner`. `AppContext` is new.

3.2.7 7. Adapter

Problem	Outside services (Gemini, Ollama, Google News, Telegram) each have their own protocol, and the core code should not depend on any of them.
Participants and roles	Targets: `LlmClient`, `NewsSource`. Adapters: `GeminiClient`, `OllamaClient`, `GoogleNewsRssSource`, `TelegramChannelSource`. Adaptees: the Gemini REST API, the Ollama `/api/chat` endpoint, an RSS feed and Telegram's public channel pages.
Files and methods	`ai/GeminiClient.java`, `ai/OllamaClient.java`, `news/GoogleNewsRssSource.java`, `news/TelegramChannelSource.java`. `complete()`, `completeJson()`, `fetch()`.
How it follows the pattern	Each adapter translates between the service's format and the application's own types (`LlmRequest`, `NewsItem`). Adding a vendor means one new class and one line in `LlmClients`. The API key goes in a request header and is never logged.
Stage 1 match	Matches. `TelegramAdapter` became `TelegramChannelSource`; `NewsFeedAdapter` became `GoogleNewsRssSource`; `OllamaClient` is the local-model adapter that replaces the Gemma command-line idea.

3.2.8 8. Factory

Problem	Creating units needs unique ids and the right battery and speed for each type. Choosing the AI client depends on configuration. Survivors must be placed reproducibly.
Participants and roles	`UnitFactory`: creates carriers, drones, robodogs and helicopters with unique ids. `LlmClients`: builds the mock, Gemini or Ollama client, wraps it with retries and builds the fallback chain. `SurvivorSpawner`: creates survivors from a seeded random generator.
Files and methods	`domain/fleet/UnitFactory.java`, `ai/LlmClients.java`, `sim/SurvivorSpawner.java`. `UnitFactory.createDrone()` and its siblings, `LlmClients.create()`, `SurvivorSpawner.spawn()`.
How it follows the pattern	Callers ask for a product and never call a constructor. These are static factory methods (a simple factory), not the Gang of Four Factory Method with a creator hierarchy.
Stage 1 match	Changed. Stage 1 had `TaskFactory` (Factory Method) for AI-generated commands and `EntityFactory` (Abstract Factory) for grid entities. Neither class exists: the compiler builds commands directly, and there are no terrain entities. The factories that remain create units, AI clients and survivors (Section 4, pattern change P2).

3.2.9 9. Template Method

Problem	Several classes follow the same steps and differ in one: the lifecycle of a command, the lifecycle of a CLI command, the tidying of a debrief.
Participants and roles	`AbstractMissionCommand`: `step()` is final and calls `onStart()` and `onStep()`. `AbstractAppCommand`: `call()` is final; it opens the application, calls `execute()`, maps exceptions to exit codes and closes it. `AbstractDebriefer`: `debrief()` is final; it calls `write()` and tidies the result.
Files and methods	`command/AbstractMissionCommand.java`, `cli/AbstractAppCommand.java`, `debrief/AbstractDebriefer.java`.

How it follows the pattern	The base class owns the order of the steps and the shared rules (start once, fail with a reason, close the database even on error). Subclasses cannot change the order. Every CLI command therefore returns the same exit codes without repeating the error handling.
Stage 1 match	Stage 1 listed Template only in the F10 traceability row. It is now used in three places.

3.2.10 10. State

Problem	An incident moves through a fixed life cycle and a unit through its working states. An illegal move, such as completing an incident that was never planned, must be impossible rather than silently accepted.
Participants and roles	`IncidentState` (NEW, VERIFIED, PLANNED, IN_PROGRESS, COMPLETED, REJECTED, DUPLICATE) and `UnitState` (IDLE, EN_ROUTE, OPERATING, RETURNING, HOLDING): enum constants that carry the transition rules. `Incident`: the context that delegates `verify()`, `plan()`, `start()`, `complete()` and `cancel()`.
Files and methods	`domain/incident/IncidentState.java`, `domain/incident/Incident.java`, `domain/fleet/UnitState.java`. `IncidentState.verify()`, `plan()`, `start()`, `complete()`, `cancel()`, `markDuplicate()`, `illegal()`.
How it follows the pattern	Each constant overrides only the transitions it allows and returns the next state; any other call throws. `MissionControl.launch()` relies on this: a plan can only start from PLANNED.
Stage 1 match	New. Stage 1 had no state pattern. It replaced status flags and `if` chains.

3.2.11 11. Builder

Problem	An incident has three required parts and several optional ones, and a stored incident must be rebuilt with its saved state. A long constructor would be easy to misuse.
Participants and roles	`Incident.Builder`: collects type, title and area up front, then optional `summary()`, `severity()`, `estimatedPeople()`, `sourceUrl()`, `reportedAt()`; `build()` validates. `IntelligenceService` and `JdbcIncidentRepository` are the clients.
Files and methods	`domain/incident/Incident.java` (nested `Builder`). `Incident.builder()`, `Builder.build()`.
How it follows the pattern	The scan builds a new incident step by step from an `Assessment`. The repository builds the same type from a database row, including its saved state, through the same builder.
Stage 1 match	New.

3.2.12 12. Repository

Problem	The rest of the program should store and find incidents without knowing whether they are in memory or in an H2 database.
Participants and roles	`IncidentRepository`: the interface. `JdbcIncidentRepository`: H2 through `java.sql` and a `ConnectionSource`. `InMemoryIncidentRepository`: no persistence.
Files and methods	`repository/*.java`, `repository/jdbc/*.java`. `save()`, `update()`, `findById()`, `findActive()`, `existsBySourceUrl()`, `deleteFinished()`, `deleteAll()`.
How it follows the pattern	`AppContext` chooses the implementation from `DB_URL`. A UNIQUE source key in the table backs up the duplicate check in the service, and a violation is translated to `DuplicateIncidentException`.

Stage 1 match	New. Stage 1 had `DatabaseTool`, `H2DatabaseManager` and `VectorSearchEngine`; the repository replaces them (Section 4, change C3).

3.2.13 13. Chain of Responsibility

Problem	The Gemini free tier allows only about 20 requests per model per day. When one model is used up, another can still answer.
Participants and roles	`FallbackLlmClient`: holds the ordered chain and the list of exhausted models. Each link is a `RetryingLlmClient` around a `GeminiClient`. `LlmException.isQuotaExhausted()`: the signal. `GeminiClient.isDailyQuota()`: tells a per-day limit from a short per-minute one.
Files and methods	`ai/FallbackLlmClient.java`, `ai/LlmException.java`, `ai/GeminiClient.java`, `ai/LlmClients.java`. `FallbackLlmClient.complete()`, `markExhausted()`, `isExhausted()`.
How it follows the pattern	A request goes to the first model that is not exhausted. A daily-quota error marks that model exhausted, logs a warning and passes the same request to the next one. Other errors are rethrown, because another model would not fix them. When every model is exhausted, the error says so and the planner or debriefer fallback takes over.
Stage 1 match	New. It was added after the trial run reached the daily limit.

3.3 Updated UML Class Diagrams

The Stage 1 diagrams showed classes such as `GridEngine`, `ActionVerifier`, `TaskFactory`, `DatabaseTool` and `VectorSearchEngine`, which do not exist in the implementation. The diagrams below were drawn from the submitted code. They show the classes and the methods that matter for each pattern; accessors and helpers are left out. Stereotypes mark the pattern role of a class.

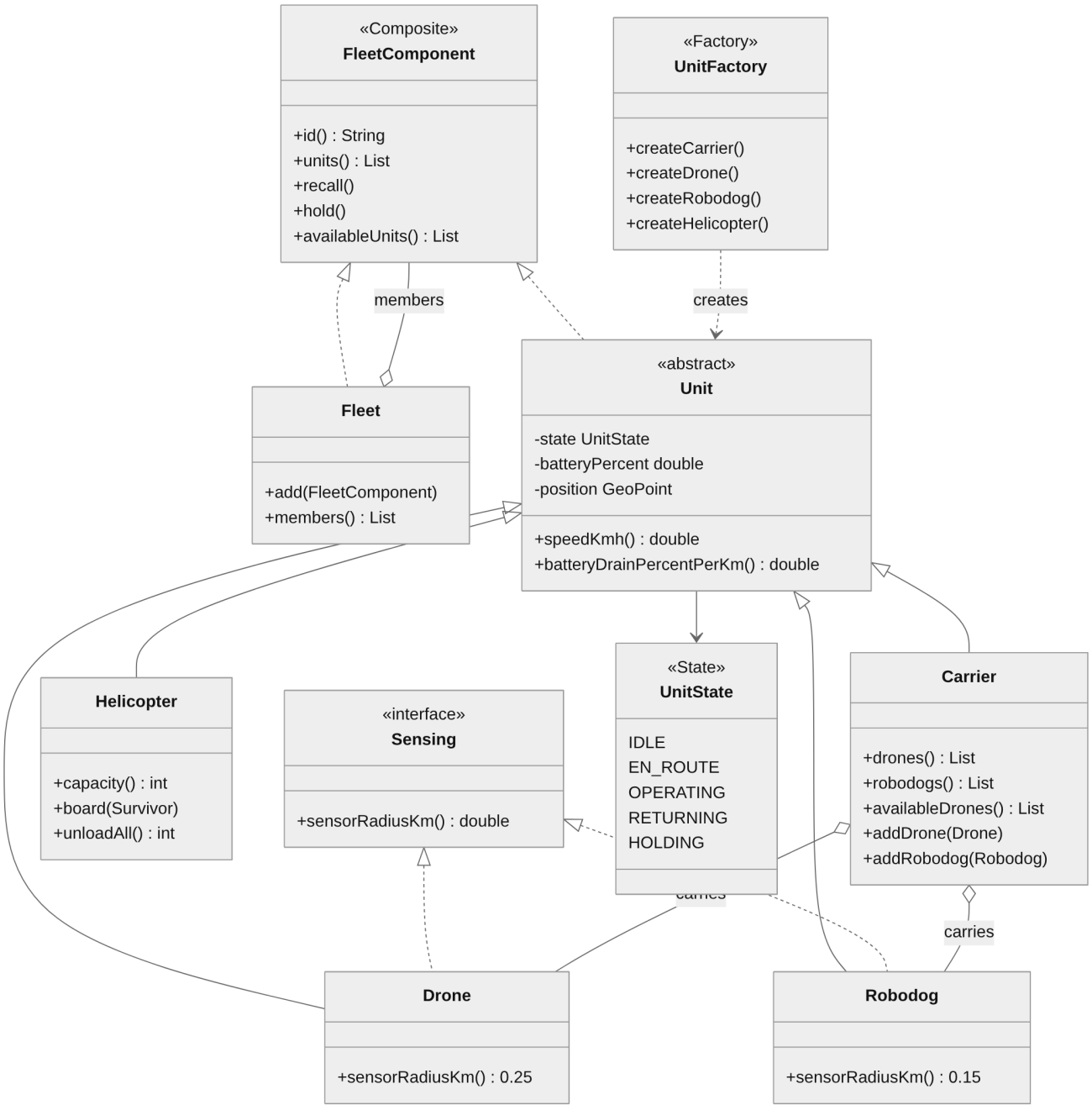


Figure 2: Class diagram 1 of 8: the fleet. Composite (`FleetComponent`, `Fleet`, `Carrier`, `Unit`), State (`UnitState`) and Factory (`UnitFactory`).



Figure 3: Class diagram 2 of 8: incidents and survivors. State (`IncidentState`), Builder (`Incident.Builder`) and Factory (`SurvivorSpawner`).

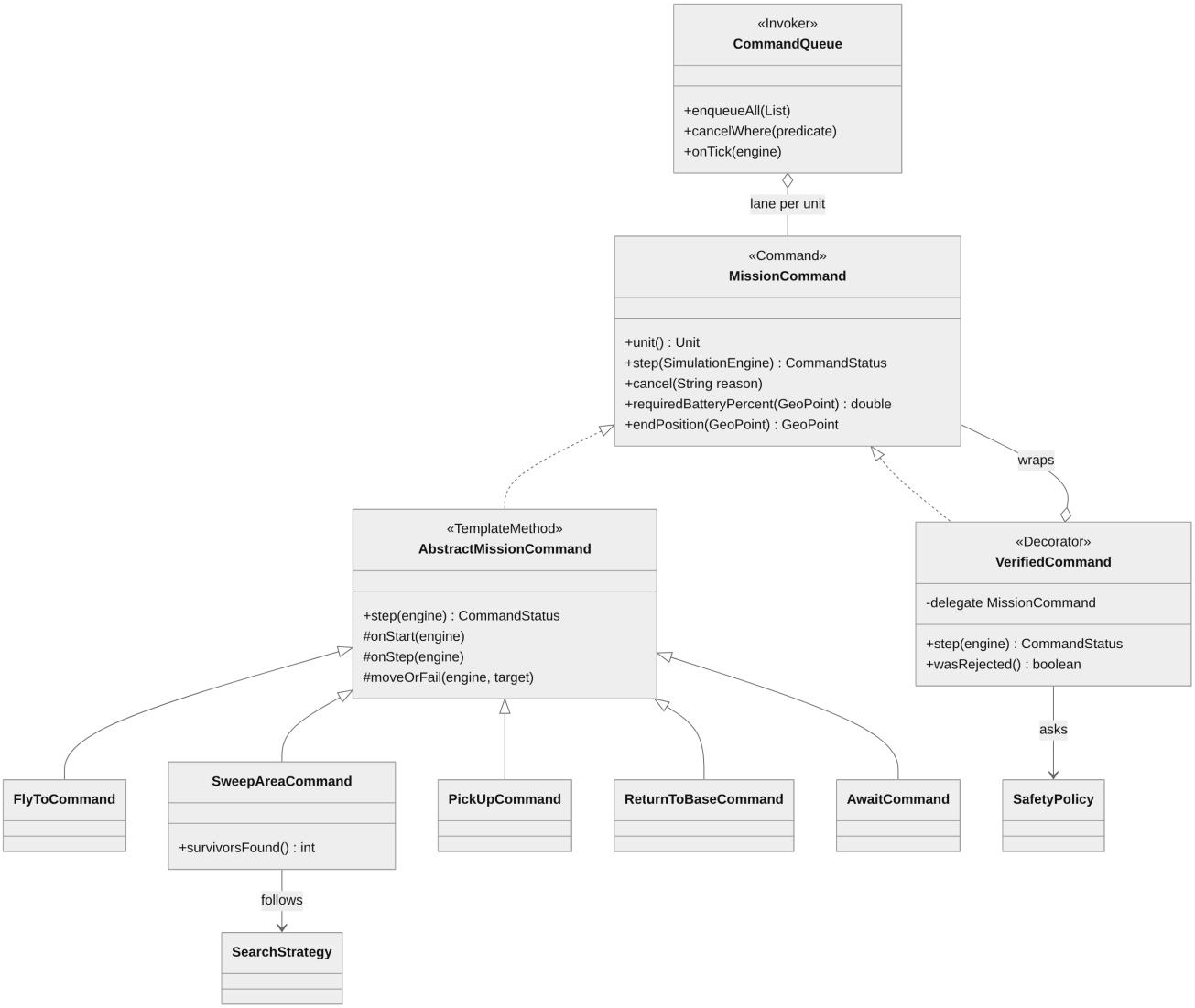


Figure 4: Class diagram 3 of 8: commands. Command, Template Method and Decorator (`VerifiedCommand`) with the per-unit `CommandQueue`.

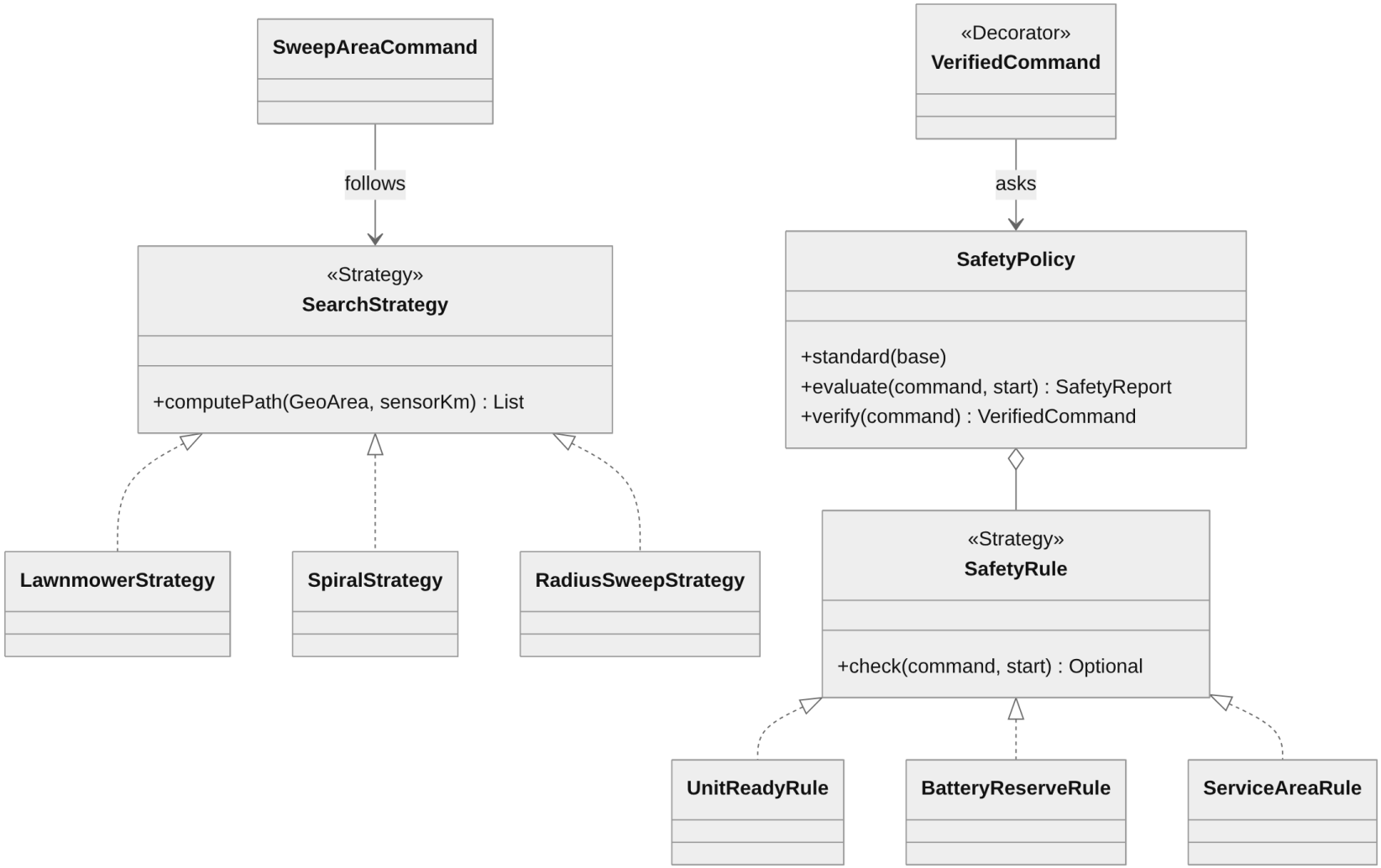


Figure 5: Class diagram 4 of 8: search strategies and safety rules. Both are Strategy families; `SafetyPolicy` combines the rules.

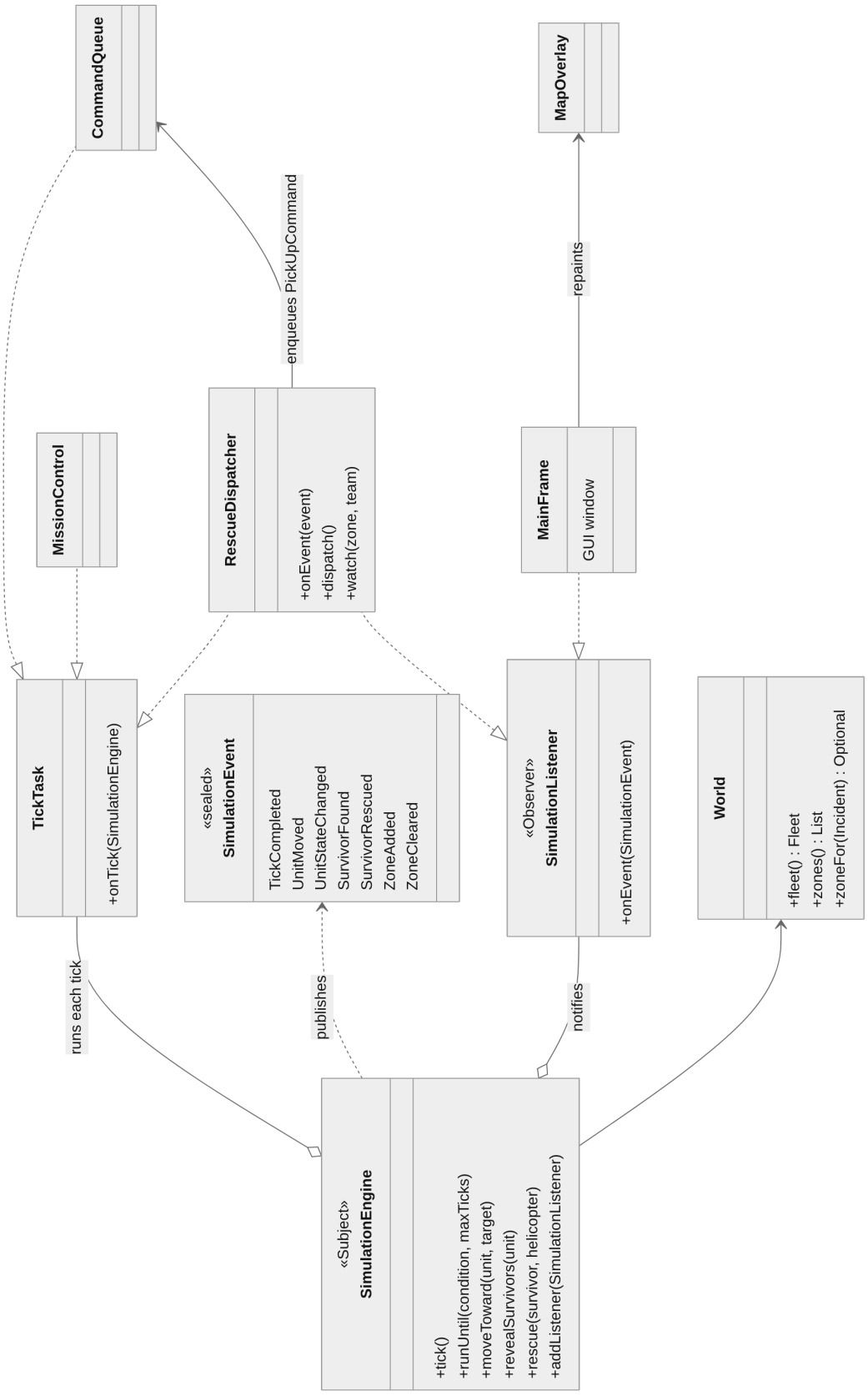


Figure 6: Class diagram 5 of 8: the simulation. `SimulationEngine` is the Observer subject; `CommandQueue`, `RescueDispatcher` and `MissionControl` run on its ticks.

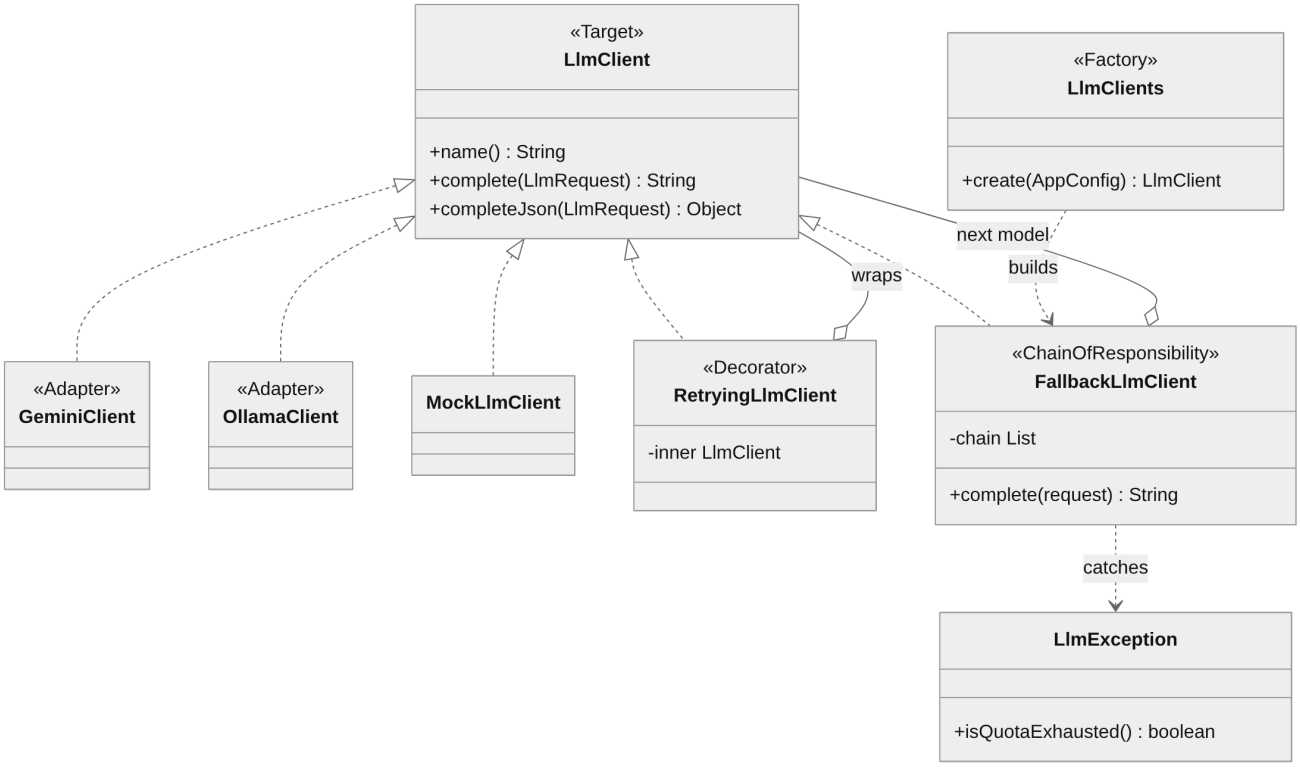


Figure 7: Class diagram 6 of 8: the AI client layer. Adapter, Decorator (`RetryingLlmClient`), Chain of Responsibility (`FallbackLlmClient`) and Factory (`LlmClients`).

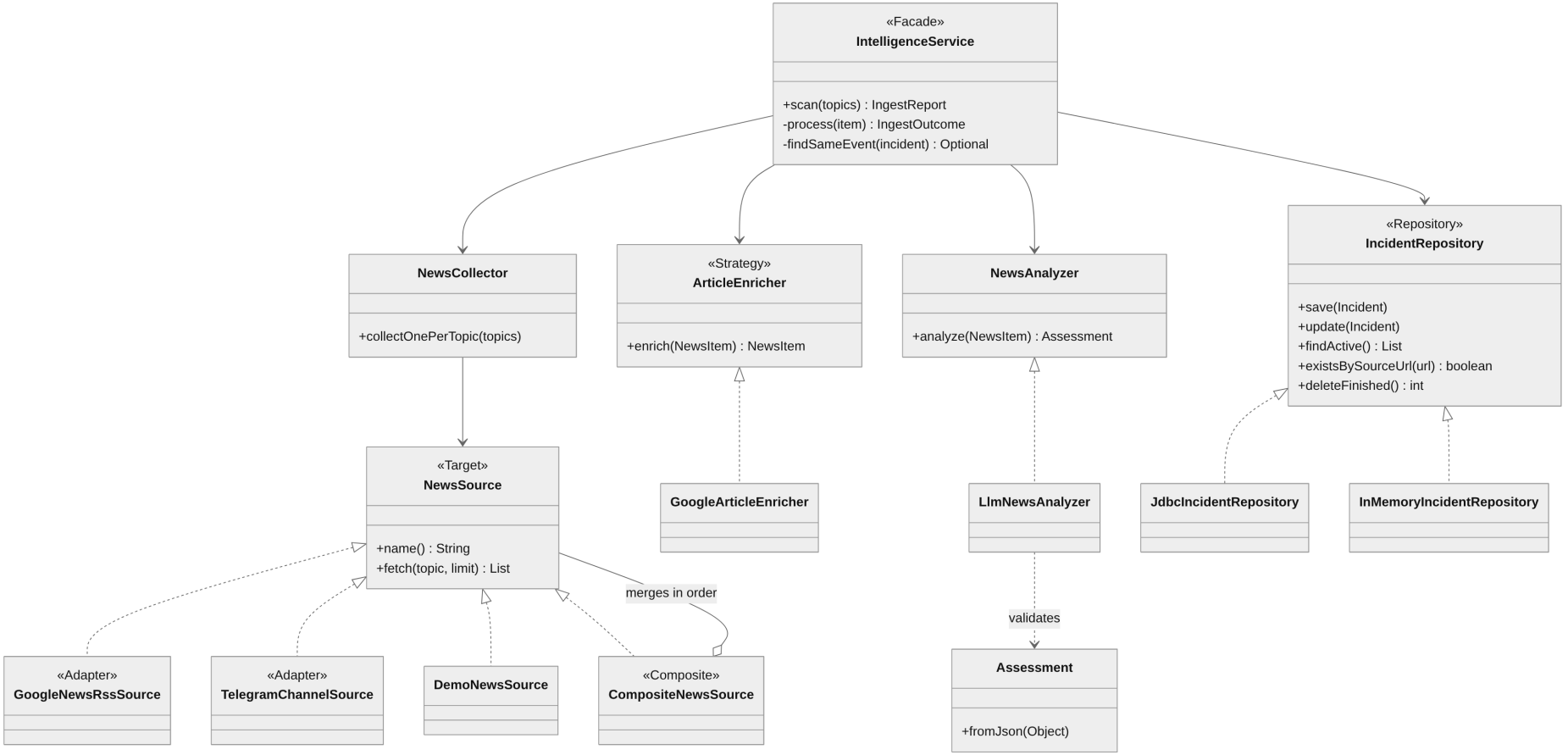


Figure 8: Class diagram 7 of 8: news, verification and storage. Adapter, Composite, Strategy, Facade (`IntelligenceService`) and Repository.

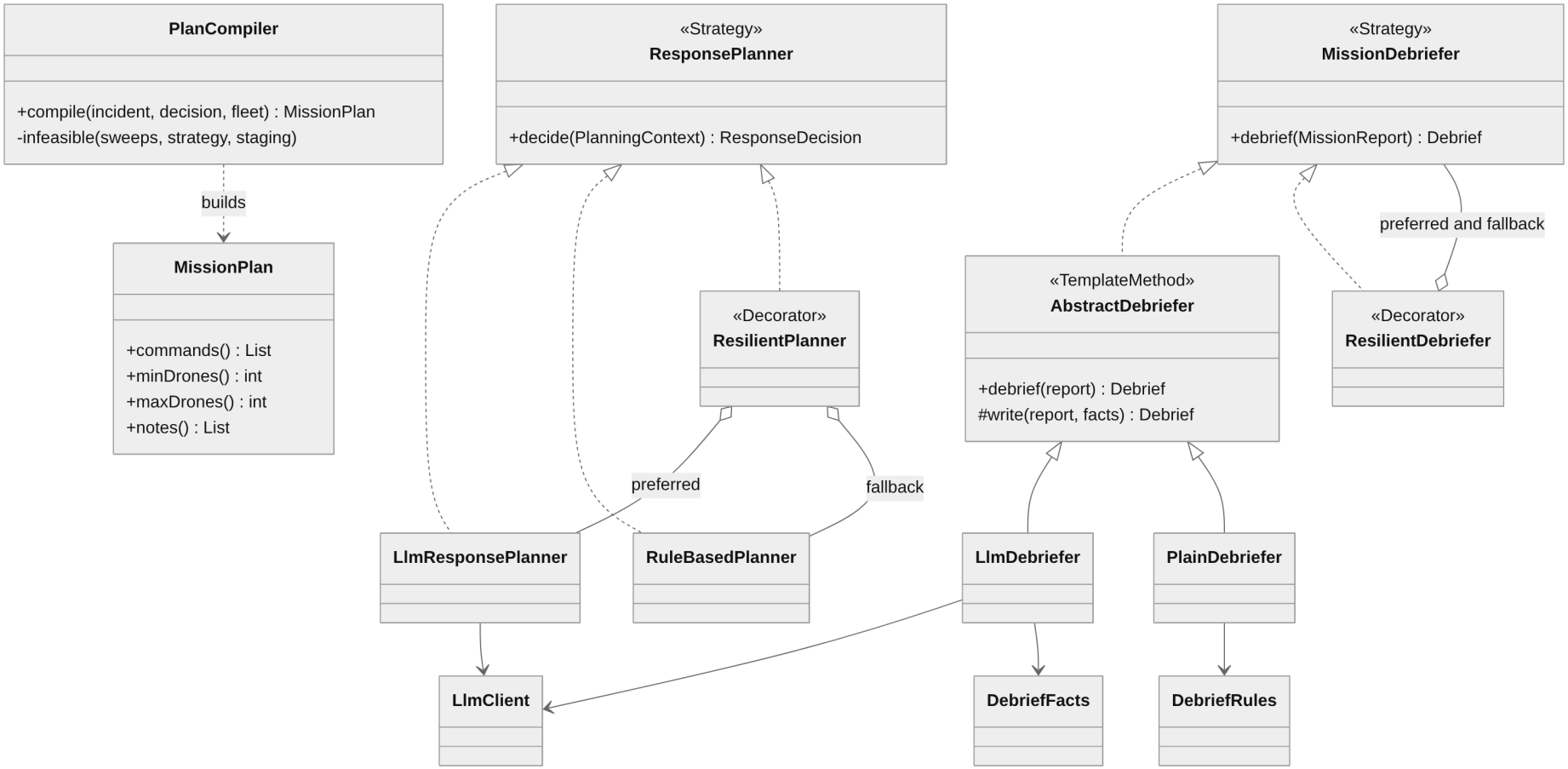


Figure 9: Class diagram 8 of 8, part 1: planning and debriefing. Strategy and Decorator for planners and debriefers, Template Method for debriefers.

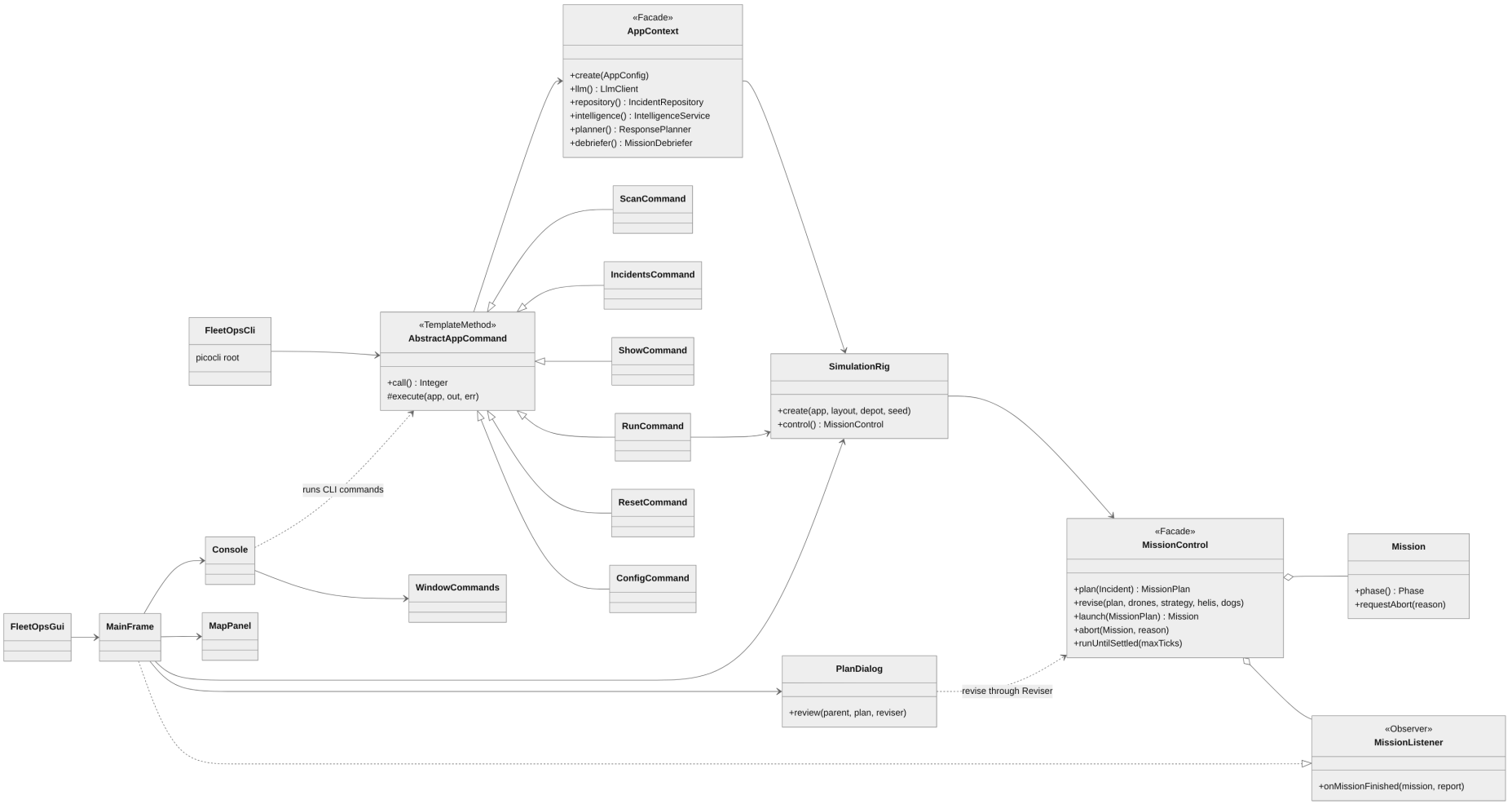


Figure 10: Class diagram 8 of 8, part 2: application, mission control, CLI and GUI. Facade, Observer and Template Method (`AbstractAppCommand`).

3.4 Updated Use-Case Diagram

Stage 1 had five technical use cases (ingest, plan, verify and execute, monitor, abort). The implementation is easier to trace with one use case per feature, so the updated diagram has UC-01 to UC-10 for F01 to F10 and UC-11 for N01. *Include* marks a step that always happens. *Extend* marks an optional step that adds to the base use case: robodogs only run when the plan asks for them, and a pick-up only follows a found survivor.

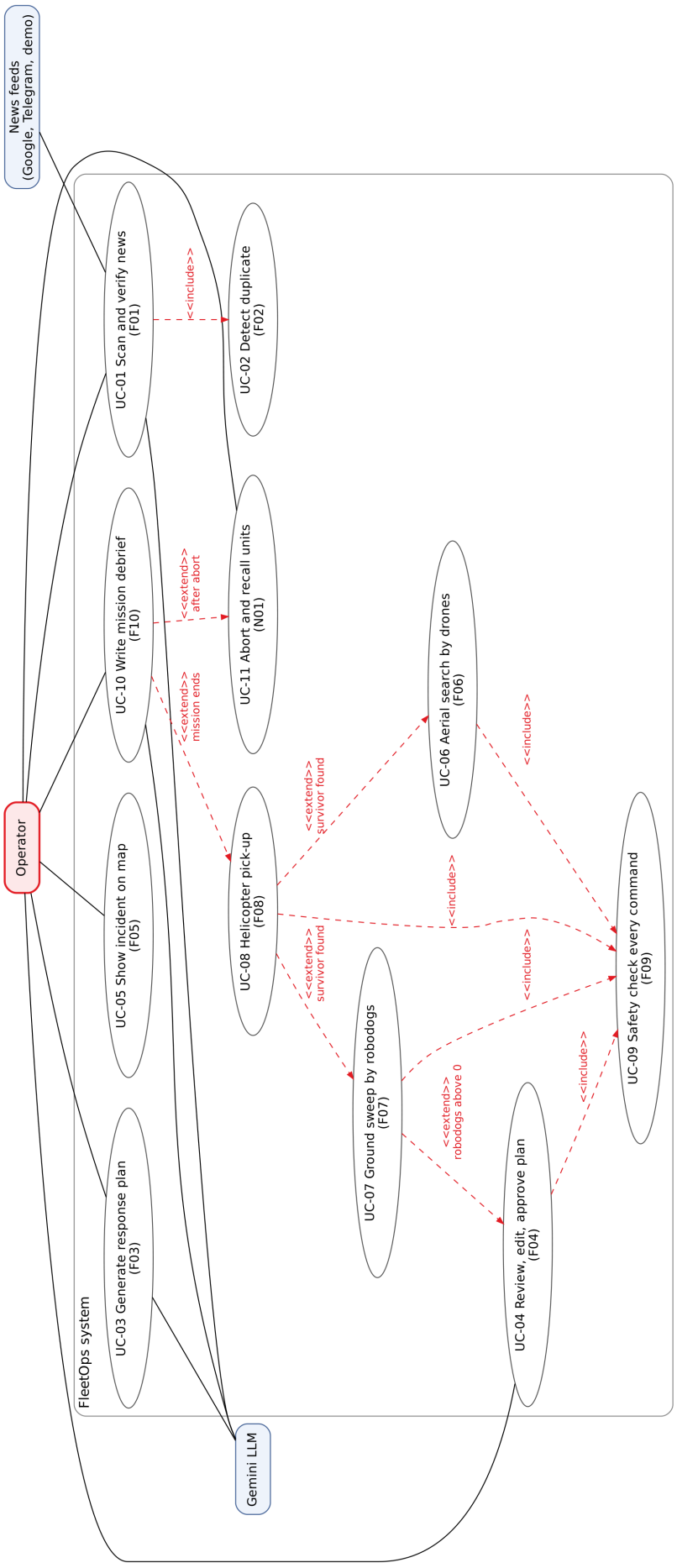


Figure 11: Updated use-case diagram. Actors: the operator, the news feeds and the Gemini LLM.

3.5 Updated Sequence Diagrams

The Stage 1 sequence diagrams (SD01 to SD04) showed `TelegramAdapter`, `DatabaseTool`, `TaskFactory` and `GridEngine`. They are redrawn for the real classes in three pairs: scanning, planning and approval, and execution with the debrief.

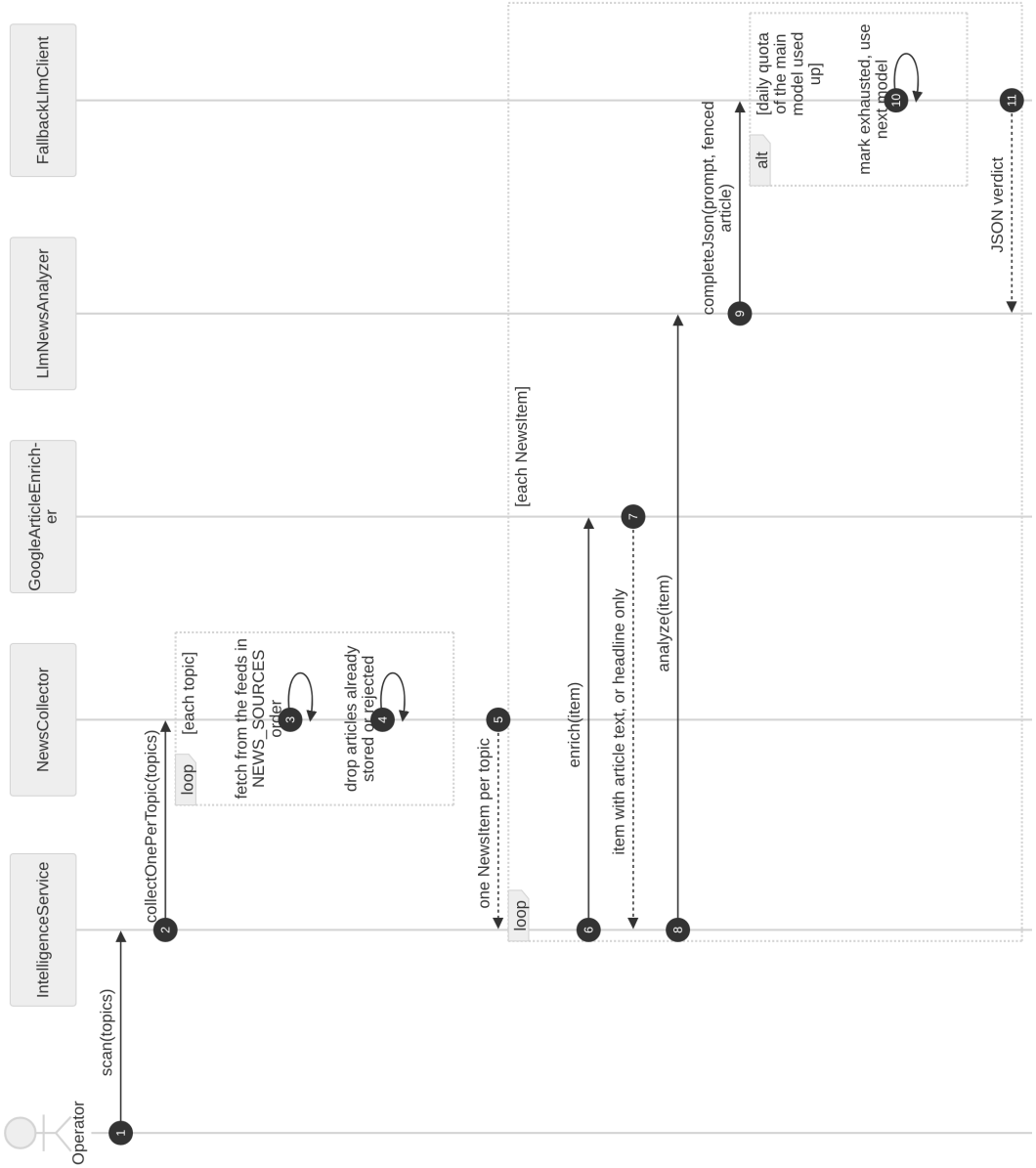


Figure 12: Sequence diagram 1a: scan, collect, enrich and analyse (F01).

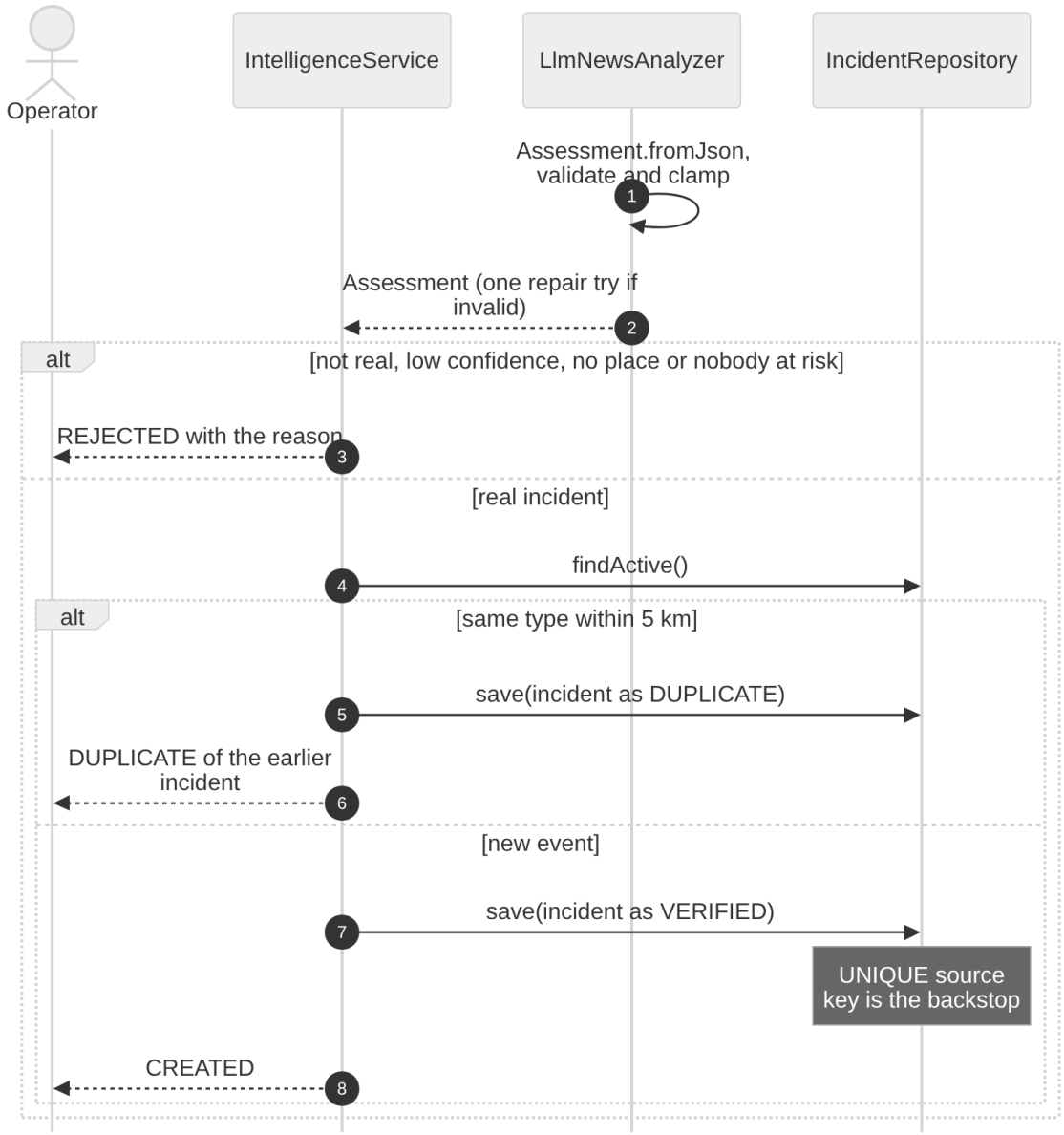


Figure 13: Sequence diagram 1b: validation, duplicate check and storage (F01 and F02).

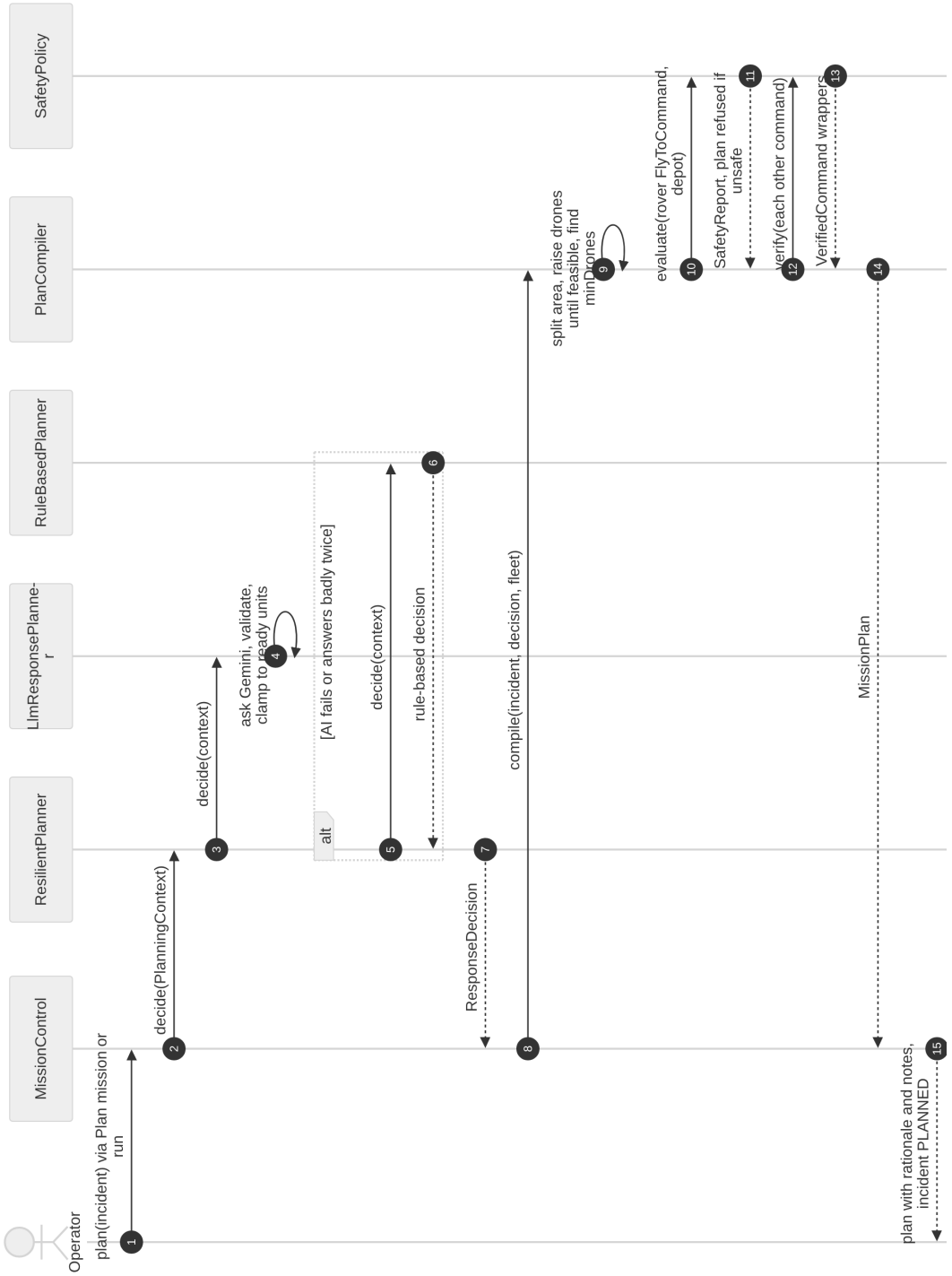


Figure 14: Sequence diagram 2a: planning, compiling and safety checking (F03 and F09).

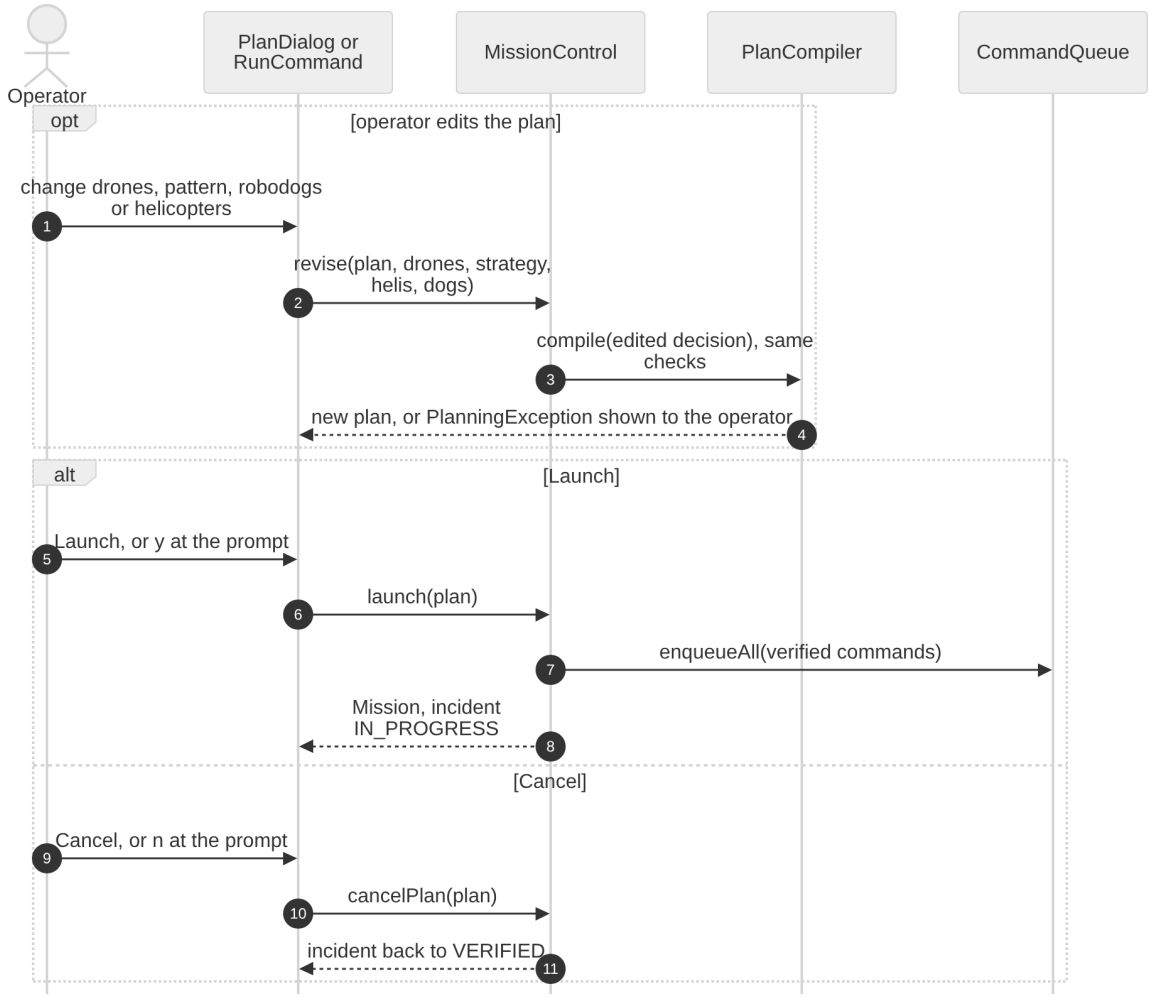


Figure 15: Sequence diagram 2b: editing, launching or cancelling the plan (F04).

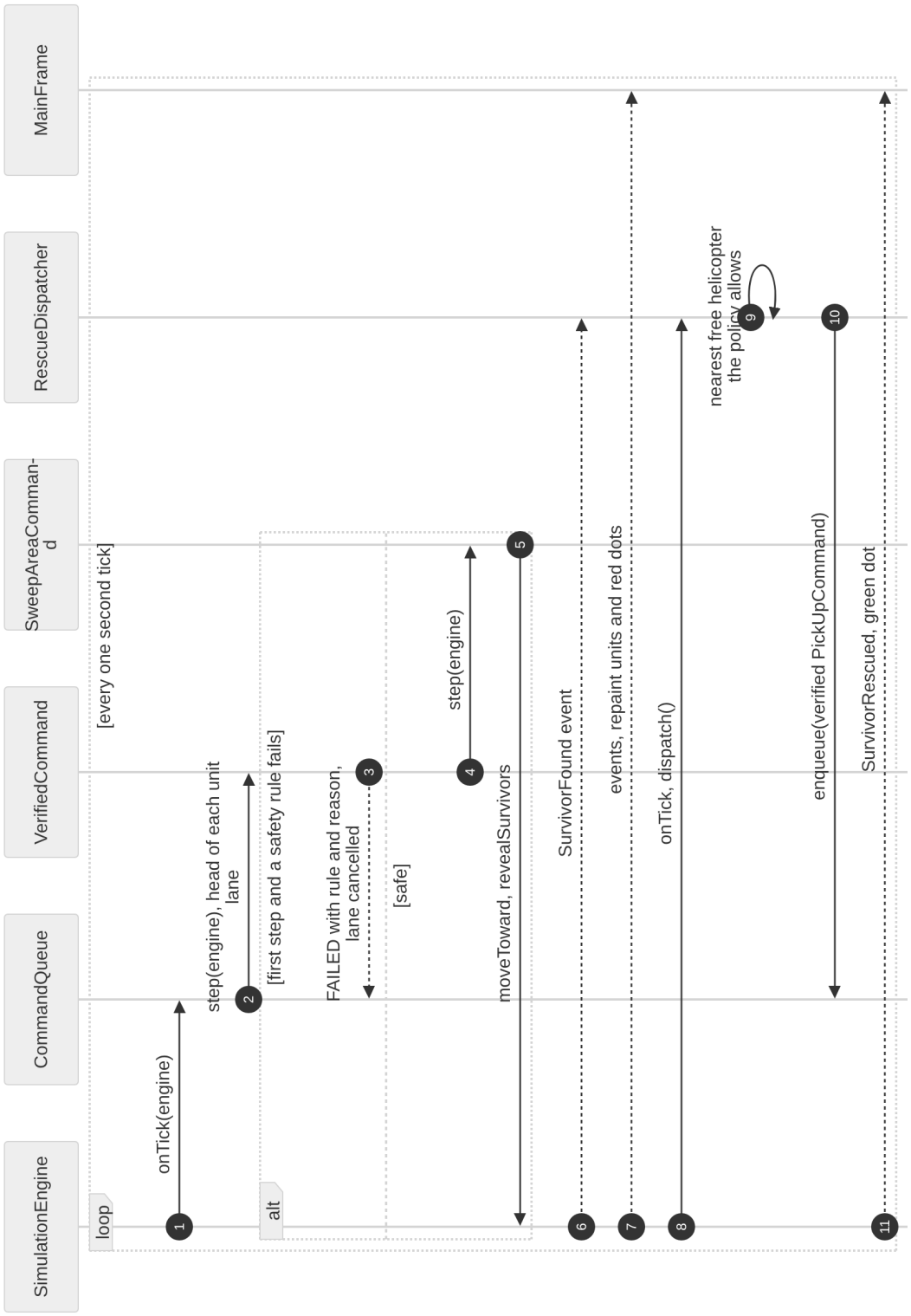


Figure 16: Sequence diagram 3a: the tick loop with safety gate, search and pick-up (F05 to F09).

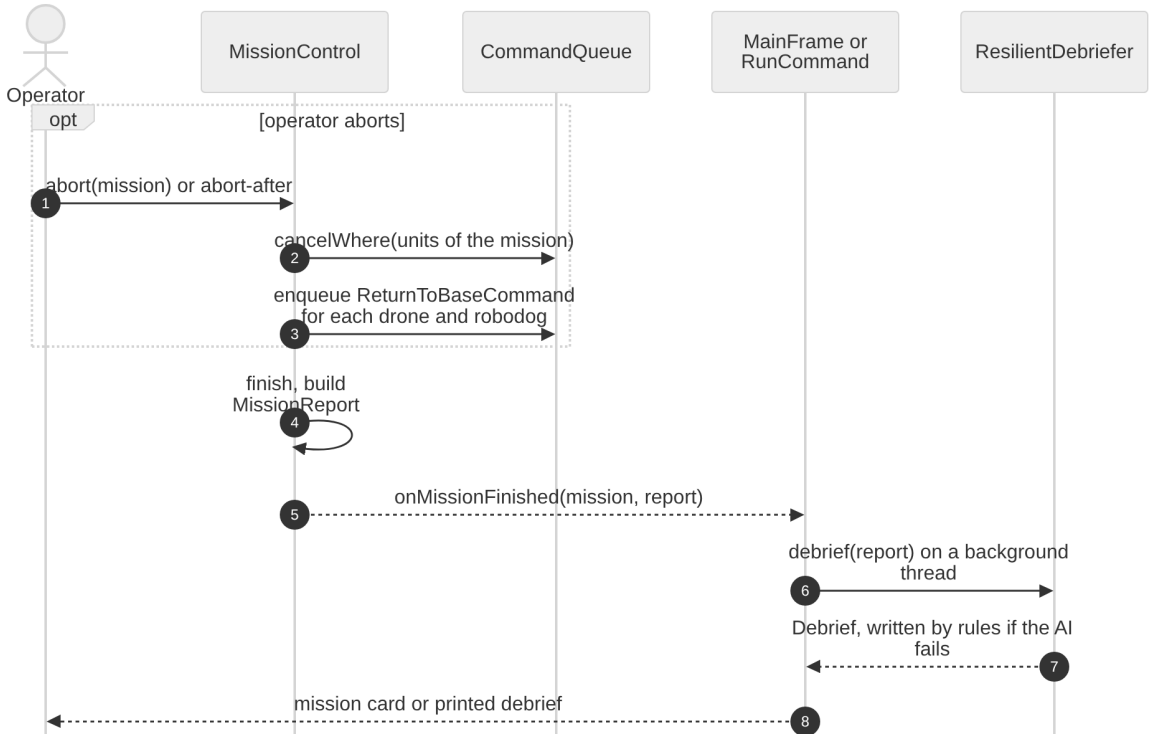


Figure 17: Sequence diagram 3b: abort, finish and debrief (N01 and F10).

4 Changes and Rationale

Every difference between the Stage 1 design and the implementation is recorded here as Original design, Implemented change, Reason, Impact. Section 4.1 covers features, 4.2 covers patterns, 4.3 maps each Stage 1 class to its implemented counterpart so a reviewer can follow a Stage 1 name into the code, and 4.4 lists additions that Stage 1 did not mention.

ID	Original design	Implemented change	Reason	Impact (use cases, classes, patterns)

4.1 Feature Changes

ID	Original design	Implemented change	Reason	Impact (use cases, classes, patterns)
C1	F01: ingest from Telegram bot, RSS or a manual paste box. CLI `scan --source telegram`.	Google News RSS, optional public Telegram channels and built-in demo stories, chosen with `NEWS_SOURCES`. The AI also reads the article text. One new item per topic per scan. No paste box.	A Telegram bot needs a token and channel rights. RSS is public. The free AI quota is small, so one item per topic keeps a scan cheap. Demo stories give an offline path.	UC-01 actor is the news feeds. `TelegramAdapter` became `TelegramChannelSource`; `NewsFeedAdapter` became `GoogleNewsRssSource`; new `ArticleEnricher`, `CompositeNewsSource`, `NewsCollector`. Patterns: Adapter, Composite, Strategy.
C2	F01 failure: show an alert and fall back to manual text entry.	A failed analysis is retried on the next scan and rejected after two failures. A dead feed is listed and the others continue.	There is no manual entry box, so the useful fallback is to keep scanning and to tell the operator why an item was dropped.	`IntelligenceService.analysisFailed()`, `ReviewLog`.
C3	F02: Gemini calls `queryDatabaseTool`; the database does a spatial and vector similarity search; hybrid.	A normalised source URL key (checked before the AI is called and enforced by a UNIQUE column) plus a same-type, 5 km distance check against active incidents. Deterministic.	Whether two reports are the same article has an exact answer. A model call per check would spend free quota and could be wrong. Vector search needs an embedding model that the free tier does not cover.	F02 is deterministic, not hybrid. `DatabaseTool`, `H2DatabaseManager`, `VectorSearchEngine` were never built. Added `IncidentRepository`, `SourceUrls`. Patterns: Repository, State.
C4	F03: Gemini returns a JSON array of commands; `TaskFactory` builds command objects; Gemini calls tools to read battery and capacity.	Gemini returns a `ResponseDecision` (drones, pattern, robodogs, helicopters, rationale). `PlanCompiler` builds and checks the commands. Fleet state goes into the prompt instead of tool calls. A rule planner takes over if the AI fails.	A model that writes commands can produce routes that are impossible. Counts and a pattern are easy to validate and clamp. Putting the fleet state in the prompt needs one request instead of a tool loop.	UC-02 is hybrid. `TaskFactory`, `PlanOrchestrator`, `TacticalPlan` replaced by `ResponsePlanner`, `PlanCompiler`, `MissionPlan`. Patterns: Strategy, Decorator, Facade.
C5	F04: toggle, reject and reorder individual tasks; `approve --plan-id`.	The operator edits drones, robodogs, helicopters and the search pattern. Each edit recompiles and rechecks the whole plan. *Back to original plan* restores the AI plan. CLI: y/n prompt and `--send-*` options.	Removing one task from a plan can leave a drone stranded or a sweep without a return. Editing the numbers keeps every plan complete and safe.	`PlanDialog`, `MissionControl.revise()`, `ResponseDecision.edited()`, `MissionPlan.minDrones()`. Patterns: Command, State, Facade.
C6	F05: 2D occupancy grid, terrain presets, walls and hazards, survivors that are stationary, drifting or fast; CLI `grid init`.	A real map (JXMapViewer2, CARTO or OpenStreetMap tiles). Incident squares come from the AI's coordinates. 3 to 20 hidden survivors per incident, seeded, stationary. No walls, no CLI grid command.	The news gives real coordinates, so a map shows them directly. A grid would need invented terrain. Fixed survivors keep a seeded run repeatable.	`GridEngine`, `GridCanvasGUI`, `EntityFactory`, `SurvivorEntity` replaced by `World`, `SimulationEngine`, `MapPanel`, `MapOverlay`, `SurvivorSpawner`. Patterns: Observer, Factory.
C7	F06: drones clear fog of war and mark obstacles for rover paths; abort the scan under 15% battery.	Drones sweep their share of the square (lawnmower or spiral) and reveal survivors. The compiler raises the drone count until every share fits the batteries. A 15% reserve and the trip home are checked before each command.	With no obstacles, there is no fog or path to clear. The real planning problem is covering the area within the battery limit.	`ReconDrone` became `Drone`; `AreaGrid`, `MissionPlan.minDrones()` added. Patterns: Command, Strategy.

ID	Original design	Implemented change	Reason	Impact (use cases, classes, patterns)
C8	F07: rover follows a cleared path, a robodog deploys at a waypoint, AI optimises the radius, a blocked path triggers a replan.	The rover drives to the south edge of the square. Each robodog is dropped along that edge and walks circles of up to 0.4 km. The AI chooses how many robodogs; the radius is a rule. No blocked-path logic.	Radius from area and battery is deterministic and can be checked. With no obstacles a blocked path cannot happen.	`TrackedRover` became `Carrier`; `RadiusSweepStrategy`; `PlanCompiler.robodogAreas()`. Patterns: Composite, Strategy.
C9	F08: landing-zone check against high obstacles; manual dispatch by clicking a found survivor.	Dispatch is automatic. `RescueDispatcher` sends the nearest free helicopter with room that the safety policy accepts. Helicopters have a capacity and unload at base.	No obstacles exist to block a landing. Automatic dispatch needs no operator at the moment a survivor appears, and the policy keeps it safe.	`EvacuateSurvivorCommand` became `PickUpCommand`; `RescueHelicopter` became `Helicopter`. Patterns: Observer, Command.
C10	F09: `ActionVerifier` checks bounds, collisions and battery; a rejected command prompts the AI to replan.	`SafetyPolicy` with `UnitReadyRule`, `BatteryReserveRule` and `ServiceAreaRule`, applied when the plan is built and again when each command starts. A rejection fails that unit's lane and shows the reason. No automatic replan.	There is no terrain, so there are no collisions to check. Checking twice covers units whose state changed after planning. An automatic replan would spend more AI quota and hide the failure from the operator.	`ActionVerifier` replaced by `SafetyPolicy`, `SafetyRule`, `VerifiedCommand`. Patterns: Decorator, Strategy.
C11	F10: `AnalyticsGUI` charts and a Gemini summary comparing AI-found targets with ground truth.	An AI debrief (summary, went well, problems, next time) built from the mission's numbers, checked by rules, with a rule-written fallback. Shown on the mission card and in the CLI. No charts.	Operators need what to change next time, and a table of numbers is already in the report. Because models invent claims, every line is validated against the numbers.	`ReportGenerator`, `AnalyticsGUI` replaced by `MissionReport`, `DebriefFacts`, `LlmDebriefer`, `PlainDebriefer`, `ResilientDebriefer`, `MissionCard`. Patterns: Template Method, Strategy, Decorator.
C12	UC-05: Override or Abort Mission, with no feature ID. Radio-link timeouts trigger return to base.	N01: abort cancels the mission's commands, holds the rover and sends drones and robodogs home. CLI `--abort-after`. No radio link is simulated.	Operators need a stop that leaves no unit stranded. A radio link adds nothing to a simulation.	New feature N01; `MissionControl.abort()`, `CommandQueue.cancelWhere()`. Patterns: Command, Facade.
C13	Primary LLM Gemini by SDK; offline Gemma by command line; tool calling.	Gemini 2.5 Flash over REST, Flash-Lite as automatic fallback, an Ollama adapter for local models, a mock for no-key runs. No tool calling.	REST needs no SDK dependency. The fallback keeps work going on the free tier.	`GeminiClient`, `OllamaClient`, `FallbackLlmClient`, `MockLlmClient`. Patterns: Adapter, Chain of Responsibility.

ID	Original pattern	Implemented pattern and classes	Problem addressed	Why the change is appropriate, effect on the architecture

4.2 Design Pattern Changes

ID	Original pattern	Implemented pattern and classes	Problem addressed	Why the change is appropriate, effect on the architecture
P1	Decorator and Safety Gate: `ActionVerifier` beside `Command` and `Robot`.	Decorator: `VerifiedCommand` wraps any `MissionCommand`. Strategy: `SafetyRules` inside `SafetyPolicy`.	Every command must pass the same checks, whoever created it.	A standalone verifier can be skipped by a caller. A wrapper cannot be, because the queue only sees `MissionCommand`. Rules can be added without touching commands.
P2	Factory Method (`TaskFactory`) and Abstract Factory (`EntityFactory`).	Factory: `UnitFactory`, `LlmClients`, `SurvivorSpawner`.	Creating units with unique ids, choosing the AI client, placing survivors.	The AI no longer emits commands and there are no terrain entities, so both Stage 1 factories had nothing to create. The new factories keep construction out of the callers.
P3	Observer: `GridObserver`, `GridEngine`, `GridCanvasGUI`.	Observer: `SimulationListener`, `SimulationEngine`, `MainFrame`, plus `MissionListener` and `CommandListener`.	GUI, dispatcher and CLI progress must react to events without the engine knowing them.	Same intent on a map instead of a grid. The dispatcher is also an observer, which lets pick-ups start the moment a survivor is found.
P4	Facade: `IntelligenceService` and `PlanOrchestrator`.	Facade: `IntelligenceService`, `MissionControl`, `AppContext`.	GUI and CLI need one entry point per area.	`MissionControl` also owns launch, abort and the queue, so both front ends share the same behaviour.
P5	Adapter: `TelegramAdapter` and `GeminiClient`.	Adapter: `GeminiClient`, `OllamaClient`, `GoogleNewsRssSource`, `TelegramChannelSource`.	Four outside services behind two interfaces.	Same pattern, more adapters. A new vendor is one class and one line in `LlmClients`.
P6	None (Template listed for F10 only).	New: State, Builder, Repository, Template Method (three uses), Chain of Responsibility, Composite for news.	Illegal state changes, validated construction, swappable storage, repeated lifecycles, quota exhaustion, merged feeds.	Each was added when the code needed it: State when incidents needed a life cycle, Repository when H2 replaced an in-memory map, Chain when the trial run used up the daily quota. The pattern count rose from 8 to 13.

4.3 Stage 1 Class to Implemented Class

Stage 1 class	Implemented class or classes	Result
`FeedScannerGUI`, `FeedScannerCLI`	`MainFrame` (Scan news), `Console`, `ScanCommand`	Replaced
`IntelligenceService`	`IntelligenceService`	Same name
`TelegramAdapter`, `NewsFeedAdapter`	`TelegramChannelSource`, `GoogleNewsRssSource`, `DemoNewsSource`, `CompositeNewsSource`	Replaced
`GeminiClient`	`LlmClient`, `GeminiClient`, `LlmNewsAnalyzer`, `LlmResponsePlanner`, `LlmDebriefer`	Split by task
`MissionIncident`	`Incident`, `Incident.Builder`, `IncidentState`	Replaced
`DatabaseTool`, `H2DatabaseManager`, `VectorSearchEngine`	`IncidentRepository`, `JdbcIncidentRepository`, `SourceUrls`	Replaced, no vector search
`MissionPlannerGUI`, `PlanApprovalGUI`, `PlanApprovalCLI`	`MainFrame`, `PlanDialog`, `RunCommand`	Replaced
`PlanOrchestrator`	`MissionControl`, `ResponsePlanner`	Replaced
`TaskFactory`, `TacticalPlan`	`PlanCompiler`, `MissionPlan`	Replaced
`CommandQueueManager`	`CommandQueue`	Renamed, per-unit lanes
`GridEngine`	`SimulationEngine`, `World`	Replaced
`GridCanvasGUI`	`MapPanel`, `MapOverlay`	Replaced

Stage 1 class	Implemented class or classes	Result
`EntityFactory`, `SurvivorEntity`, `ObstacleCell`	`SurvivorSpawner`, `Survivor`; no obstacle class	Reduced
`Robot`, `ReconDrone`, `TrackedRover`, `Robodog`, `RescueHelicopter`	`Unit`, `Drone`, `Carrier`, `Robodog`, `Helicopter`	Renamed
`Command`, `ScanAreaCommand`, `TraversePathCommand`, `EvacuateSurvivorCommand`	`MissionCommand`, `SweepAreaCommand`, `FlyToCommand`, `PickUpCommand`, `ReturnToBaseCommand`, `AwaitCommand`	Renamed, two added
`SearchStrategy`, `LawnmowerScanStrategy`, `RadiusSweepStrategy`	`SearchStrategy`, `LawnmowerStrategy`, `SpiralStrategy`, `RadiusSweepStrategy`	Extended
`ActionVerifier`	`SafetyPolicy`, `SafetyRule` (three rules), `VerifiedCommand`	Replaced
`ReportGenerator`, `AnalyticsGUI`	`MissionReport`, `DebriefFacts`, `LlmDebriefer`, `PlainDebriefer`, `ResilientDebriefer`, `MissionCard`	Replaced

4.4 Additions Not in Stage 1

- **Article enrichment.** Google News links point to a redirect page, so the AI first saw almost nothing. `GoogleArticleEnricher` finds the real article and gives the model its paragraphs.
- **Model fallback chain.** Needed after the free-tier daily limit stopped a trial run.
- **Embedded console.** A terminal panel in the GUI runs the same commands as the CLI against the same database, plus 13 window commands (`plan`, `edit`, `launch`, `cancel`, `abort`, `debrief`, `focus`, `speed`, `fleet`, `layout`, `help`, `clear`, `exit`).
- **Plan notes and feasible range.** The plan explains when it raised a drone count or left a robodog behind and shows the fewest drones that can do the job.
- **Layout presets, speed control and incident cards** with search, filters and sorting in the GUI.

5 AI Tools and Collaboration Log

Two AI systems are involved. Claude Code is the development tool I used to write the code. Gemini is the model that runs inside FleetOps; its calls came from my testing and from recording the demonstration. Both are logged here.

5.1 Tools, Models, Tasks and Cost

Tool or platform	Model	Features, classes or development tasks	Total tokens	Money cost
Claude Code (Anthropic, desktop app)	Claude Opus 5.5 and Claude Sonnet 5.5	Writing and revising the Java code in all 30 code commits: domain, fleet, simulation, commands, safety gate, news adapters, AI clients, intelligence service, planner and compiler, mission control, storage, CLI, GUI, debrief. Also debugging from the output of my trial runs, the throwaway check programs, the README, the architecture notes, the diagrams and this report.	About 1,170,000 tokens in total, read from the Claude usage page (input and output are not shown separately)	$30 per month subscription, no per-token charge
Gemini API, Google AI Studio free tier	`gemini-2.5-flash`	Runtime LLM for F01 verification, F03 planning and F10 debrief, during development testing and the demonstration recording	76k input and 91k output	$0 (free tier)

Tool or platform	Model	Features, classes or development tasks	Total tokens	Money cost
Gemini API, Google AI Studio free tier	`gemini-2.5-flash-lite`	Automatic fallback model when the Flash daily allowance was used up	29.4k input and 2.3k output	$0 (free tier)
Gemini total		129 requests	105.4k input and 93.3k output	$0

5.2 Gemini Usage Readings

I read the usage dashboard three times during development. The requests are not split by model. The token figures are in thousands.

	Reading 1	Reading 2	Reading 3	Total
Requests, all models	55	31	43	129
Flash input tokens	13k	29k	34k	76k
Flash output tokens	22k	21k	48k	91k
Flash-Lite input tokens (two readings)	16k	13.4k		29.4k
Flash-Lite output tokens (two readings)	1.7k	0.6k		2.3k

5.3 Division of Work

- **What I did.** I wrote the Stage 1 design that the code follows and decided every departure from it. I ran every trial run and pasted the output back to Claude Code. I chose to stay on the free tier, which led to the model fallback. I recorded and edited the video. I made every git commit and push myself, and I kept the API key in a local `.env` file.
- **What Claude Code did.** It wrote the code, the check programs and the documents from my instructions, explained options, and proposed fixes for the problems I reported. I told it not to change code until I said so, and it waited for that instruction before editing.

#	My request	What Claude Code did	What I checked or changed	Result

5.4 Representative Interactions

Each row is a real exchange. My requests are quoted as I typed them.

#	My request	What Claude Code did	What I checked or changed	Result
1	After a scripted CLI and GUI trial run I pasted all the output and wrote "fix all issues".	Listed the problems in the output and fixed six of them: region-wide stories, incidents with nobody at risk, the minimum drone count, debrief wording, an overlong URL in `show`, and the planner prompt.	I repeated the trial steps and asked "how to test if the updates work", then ran the new checks. One reported problem could not be reproduced, so it was left unchanged instead of guessed at.	Commit `3cd8158`
2	"i want to stay on free. dont code yet but is it possible we setup so that we start with 2.5 and if limit then we switch to the flash lite one automatically"	Explained a chain of models, how to tell a daily limit from a short per-minute one, and waited. After I said "go", it wrote `FallbackLlmClient`, `GeminiClient.isDailyQuota()` and the `LLM_FALLBACK_MODELS` setting.	I ran `config` to see the chain and ran a scan with the new build.	Commit `798b0ea`
3	"its good . it says send 2 drones instead of 4 but it wouldnt let me change the number from 4 to 2 and kept reverting to 4 when i tried"	Traced it to the dialog having no idea of the fewest drones the batteries allow. Added `minDrones` and `maxDrones` to `MissionPlan`, computed in `PlanCompiler`, and made the spinner floor follow them, with a tooltip.	Re-tested the dialog in the GUI by lowering and raising the count.	Commit `3cd8158`
4	I reported that the debrief text sometimes disagreed with the report numbers.	Added the units sent and available to the facts header, rules in the prompt (minutes include the flight home, never write "aborted because"), and `DebriefRules.advicePossible()` to drop advice that needs units that do not exist.	Read debriefs from a full mission and an aborted one in the CLI and the GUI.	Commits `1455947` and `3cd8158`
5	My screen recording showed the map window as black.	Suggested a Java graphics flag. With it the window did not open at all.	I reverted the flag and kept the existing Maven option, which opened the window.	No code change. Shows an AI suggestion that failed when tested.
6	I attached a README from another project and asked for the same format, "no mention of the course", as a regular GitHub project.	Rewrote the README with a flow diagram, a configuration table and usage, and updated `docs/ARCHITECTURE.md`.	I left the demo image as a placeholder and kept the course out of the text.	Commit `5c70c6b`

5.5 Keeping the AI Under Control

- **Secrets.** The key lives in `.env`, which `.gitignore` excludes. `.env.example` holds only placeholders. The `config` command never prints a key, and the Gemini client sends it in a request header.
- **Review.** Claude Code did not commit anything. I made each commit and push, and I ran the trial runs that found the defects in Section 6.
- **The AI inside the product is untrusted.** Its answers are validated and clamped, article text is fenced in the prompt, and the deterministic safety gate has the last word before any unit moves (Sections 1.3 and 3).

6 Development and Verification Summary

6.1 Weekly Commit Summary

All commits are on the `main` branch of https://github.com/abhisheksaraff/fleet-ops. Each hash below links to its commit page (`https://github.com/abhisheksaraff/fleet-ops/commit/<hash>`). Times are Toronto local time. Commits that only change documents or move files are not counted and are listed at the end of this subsection.

Period	Dates (Toronto time)	Required code commits	Code commits	Result
1	October 20 to 26, 2026	5	5	Met
2	October 27 to November 2, 2026	5	5	Met
3	November 3 to 9, 2026	5	20	Met

Period 1: October 20 to 26

#	Commit	Date and time	Message
1	2ae1841	Oct 24, 14:30	build: set up Maven project with config loading and GUI/CLI entry points
2	2ed37e9	Oct 24, 15:21	feat(domain): add geo model, incident lifecycle and survivors
3	8b87283	Oct 25, 12:03	feat(fleet): add unit hierarchy, composite fleet and unit states
4	9cc2282	Oct 25, 13:15	feat(sim): add tick-based simulation engine with observer events
5	b034eca	Oct 25, 13:44	feat(command): add step-based mission commands and command queue

Period 2: October 27 to November 2

#	Commit	Date and time	Message
1	6a9fe06	Nov 2, 03:07	feat(safety): add safety rules and VerifiedCommand decorator
2	87a9434	Nov 2, 18:50	feat(news): add news source adapters and incident repository
3	2066955	Nov 2, 19:11	feat(ai): add LLM client interface with Gemini, Ollama and mock
4	19762d1	Nov 2, 21:22	feat(intel): verify and extract incidents from news with the LLM

#	Commit	Date and time	Message
5	9d07675	Nov 2, 21:35	feat(plan): add AI response planner and mission plan compiler

Period 3: November 3 to 9

#	Commit	Date and time	Message
1	3078efb	Nov 3, 22:30	feat(mission): add MissionControl facade and rescue dispatcher
2	bef354b	Nov 3, 22:55	feat(storage): add JDBC incident repository and news/DB configuration
3	738214c	Nov 3, 23:07	feat(cli): add scan, incidents, show and run commands
4	d8063d9	Nov 3, 23:20	feat(gui): add map-based mission control window
5	10feec8	Nov 3, 23:31	feat(storage): add incident deletion with CLI reset and GUI clear
6	45fbd9b	Nov 5, 23:19	feat(gui): dark theme and resizable layout
7	4880f5a	Nov 5, 23:27	feat(gui): incident cards with search, filters and sorting
8	c3e29aa	Nov 5, 23:37	feat(gui): dark map, round markers, mission card and plan dialog
9	1be5bb2	Nov 5, 23:43	feat(gui): pixel-art units and survivors with animated rotors
10	eb2c6c1	Nov 5, 23:51	feat(gui): embedded console with banner, history and tab completion
11	2b953d7	Nov 5, 23:57	feat(gui): console commands that drive the map
12	090ba2a	Nov 7, 03:02	fix(plan): refuse a plan whose base is out of the carrier's range
13	321e566	Nov 7, 03:02	feat(gui): survey-sheet redesign with CARTO dark tiles
14	1455947	Nov 7, 03:15	feat(report): AI-written mission debrief
15	eacd259	Nov 7, 03:25	feat(plan): operator edits the plan before approval
16	d361f2c	Nov 7, 03:37	feat(fleet): robodogs carried by rovers with a radius sweep
17	a431fae	Nov 7, 03:46	feat(mission): deploy robodogs in missions, GUI and CLI
18	da9e1de	Nov 7, 22:34	feat(mission): Improve AI planning, debriefs and news input
19	3cd8158	Nov 8, 00:39	fix(plan): Fix issues found with Gemini scan, plan and debrief
20	798b0ea	Nov 8, 00:47	fix(ai): Add automatic fallback between Gemini models

Commits not counted. 6c3308e (architecture overview and README), e8e2aae (moves Stage 1 files), 6cc05d7 (README) and 5c70c6b (README and architecture notes). The first commit, e876fe5 of October 10, belongs to Stage 1. The repository holds 35 commits in total: 1 from Stage 1, 30 code commits and 4 documentation commits.

Check	What it covered	Outcome

6.2 Checks Performed

Stage 2 asks for basic verification. The repository has no committed tests, because the full deterministic test suite belongs to Stage 3. What I did instead is listed here.

Check	What it covered	Outcome
Scripted CLI trial run	A written script of about twenty steps: `config`, `reset`, `scan` with several editions, `incidents`, `show`, `run --dry-run`, a run that I approved, one I declined, one I aborted, `reset -f`. I compared each result with what the script said to expect.	Found 7 problems (Section 6.3); 6 fixed, 1 could not be reproduced
Scripted GUI trial run	Scan, select, plan, edit the drone count, launch, watch the search and pick-up, abort, read the debrief, clear finished incidents.	Found the drone spinner problem and the debrief wording problems
Throwaway check programs	Twelve small Java programs, not committed, that call the real classes. They cover planning and the compiler, mission control, debrief rules and prompts, the fallback chain, Gemini error classification, the plan dialog and CLI output. The last three programs alone hold 82 assertions.	All pass on the final commit
Build check	`mvn -q compile` and `mvn -q exec:java` on Java 21.	Builds and starts
Demonstration rehearsal	I ran the full demonstration script before recording and restarted the recording when the map showed black.	Final recording uses the corrected build
Secret scan	Searched all 35 commits for the Google key pattern (`AIza` followed by 20 or more characters), checked that `.env` was never tracked, and read `.env.example`.	0 matches; `.env` never tracked; `.env.example` has placeholders only

6.3 Defects Found and Corrected

#	Problem	Cause	Fix	Commit
D1	A plan was produced for an incident too far from the rover's base, and the first command failed only at launch.	The range check ran when commands started, not when the plan was built.	`PlanCompiler` evaluates the rover's drive with the safety policy and refuses the plan with the distance.	090ba2a
D2	The AI judged Google News stories from a headline alone, and demo stories took the slots that real news should have had.	Google links lead to a redirect page; feeds were merged in an arbitrary order.	`GoogleArticleEnricher` downloads the real article; `CompositeNewsSource` merges feeds in the order of `NEWS_SOURCES`.	da9e1de
D3	A story about a whole province or country was accepted as one incident.	The prompt did not say that an incident must be a place.	Prompt rule: not real if the story covers a whole country, province, state or region.	3cd8158
D4	An incident with nobody at risk was stored and could be planned.	No check on the number of people.	`IntelligenceService.process()` rejects `estimatedPeople` below 1; the prompt asks for at least 1 for a real incident.	3cd8158
D5	In the plan dialog I typed 2 drones and the value reverted to 4.	The compiler raised the count to what the batteries allow, but the dialog did not know the minimum.	`MissionPlan.minDrones()` and `maxDrones()`; the spinner floor follows them and a tooltip explains.	3cd8158

#	Problem	Cause	Fix	Commit
D6	Debrief text disagreed with the numbers: minutes without the flight home, "aborted because" wording, advice to send units that do not exist.	The model only saw the counts, not the limits.	Facts header adds fewest drones and units available; prompt rules; `DebriefRules.advicePossible()`; report text says minutes include the flight home.	3cd8158
D7	`show` printed a very long source address across the screen.	No limit on the field.	`CliFormat.clip(source, 80)`.	3cd8158
D8	The free daily allowance of Gemini 2.5 Flash ended a scan with an error.	A daily limit was treated like a short per-minute one.	`GeminiClient.isDailyQuota()` marks daily limits as not retryable; `FallbackLlmClient` moves to the next model.	798b0ea
D9	A report that an incident vanished after `reset -f` following an abort.	Could not be reproduced: an aborted CLI run leaves the incident VERIFIED, and `reset -f` removes only finished incidents.	No change.	None

6.4 Remaining Limitations and Stage 3 Work

The limitations in Section 1.6 still stand. For Stage 3 the remaining work is a committed deterministic test suite (state transitions, command lanes, the safety rules, the compiler's feasibility search, the repository and the validation of AI answers) and the agent validation that the Project Instructions require. The same throwaway checks are the starting point for the tests.

7 Demonstration Index

All demonstrations are in one recording: https://www.youtube.com/watch?v=1e3nmwXIGqg. Each segment starts with a title card that names the feature ID and feature, shows how the feature is started and what it needs as input, runs it in the actual application, and says where the AI is involved. The CLI segments run in a terminal and the GUI segments run in the map window, so both interfaces are shown. The links below open the video at the start of the segment.

7.1 Feature Index

ID	Feature	Link and time	Interface	Where the AI is involved
F01	Intelligence feed ingestion and parsing	0:15	CLI	The model reads each story and returns the structured verdict
F02	Deduplication of incidents	0:29	CLI	None; a database key and a distance check
F03	Autonomous task plan generation	0:46	CLI	The model chooses drones, pattern, robodogs and helicopters and explains why
F04	Plan approval and manual override	0:57 (CLI), 1:56 (GUI)	CLI, GUI	None; the operator decides and edits
F05	Environment and survivor generator on a map	2:43	GUI	None
F06	Aerial reconnaissance by drones	2:53	GUI	The plan's drone count and pattern come from the model

ID	Feature	Link and time	Interface	Where the AI is involved
F07	Ground sweep with rover and robodogs	3:06	GUI	The number of robodogs comes from the model
F08	Helicopter extraction	3:23	GUI	The number of helicopters comes from the model
F09	Deterministic safety gate	1:24	CLI	None; shows a command refused by the rules
F10	Post-mission debrief	1:38 (CLI), 3:49 (GUI)	CLI, GUI	The model writes the debrief from the mission's numbers
N01	Mission abort and recall	1:38	CLI	None

7.2 Segment Timeline

Time	Segment	Interface	What the segment shows
0:00	Title and configuration	CLI	The `config` output: provider, model, fallback chain and news sources. Keys are shown as set or not set, never as values.
0:15	F01 feed ingestion	CLI	`scan`: items read, the verdict for each, and incidents created or rejected with reasons.
0:29	F02 deduplication	CLI	A second `scan` that skips stored articles, then `incidents`.
0:46	F03 plan generation	CLI	The plan the model chose, with its reasoning.
0:57	F04 approval, CLI	CLI	The plan and the approval prompt. Nothing runs until the operator approves.
1:24	F09 safety gate	CLI	A command refused by a safety rule, with the reason.
1:38	F10 and N01	CLI	A mission stopped early, the units recalled and the debrief printed.
1:56	F04 approval, GUI	GUI	The plan dialog: change drones, robodogs, helicopters or pattern, restore the original, launch.
2:43	F05 map	GUI	The map, the incident square and the units.
2:53	F06 drones	GUI	Drones sweeping their cells and survivors appearing as red dots.
3:06	F07 robodogs	GUI	The rover dropping robodogs that walk circles.
3:23	F08 helicopters	GUI	Helicopters picking up found survivors; rescued survivors turn green.
3:49	F10 debrief, GUI	GUI	The debrief on the mission card.
4:34	Wrap-up		Summary of what was shown.

Error and alternative behaviour is shown for F09 (a command refused), N01 (a mission stopped early) and F04 (the operator's choice before launch).

Music credit. Music by prettyjohn1 from Pixabay, https://pixabay.com/users/prettyjohn1-54616349/.

A Appendix: Traceability, Configuration and Usage

A.1 Stage 1 Requirement to Running Demonstration

Each row follows one feature from its Stage 1 reference through the use case, the main classes and methods and the patterns to the demonstration.

ID	Use case	Main classes and methods	Patterns	Video
F01	UC-01	`ScanCommand`, `IntelligenceService.scan()`, `NewsCollector`, `LlmNewsAnalyzer.analyze()`, `Assessment.fromJson()`	Adapter, Composite, Strategy, Facade, Chain	0:15
F02	UC-02	`SourceUrls.normalize()`, `IncidentRepository.existsBySourceUrl()`, `IntelligenceService.findSameEvent()`	Repository, State	0:29
F03	UC-03	`MissionControl.plan()`, `LlmResponsePlanner.decide()`, `PlanCompiler.compile()`	Strategy, Decorator, Facade	0:46
F04	UC-04	`PlanDialog.review()`, `MissionControl.revise()`, `launch()`, `cancelPlan()`	Command, State, Facade	0:57, 1:56
F05	UC-05	`MapPanel`, `MapOverlay`, `SimulationEngine.tick()`, `SurvivorSpawner.spawn()`	Observer, Factory	2:43
F06	UC-06	`SweepAreaCommand`, `SearchStrategy.computePath()`, `AreaGrid.assign()`	Command, Strategy, Template Method	2:53
F07	UC-07	`Carrier`, `Robodog`, `RadiusSweepStrategy`, `PlanCompiler`	Composite, Strategy, Command	3:06
F08	UC-08	`RescueDispatcher.dispatch()`, `PickUpCommand`, `Helicopter.board()`	Observer, Command	3:23
F09	UC-09	`SafetyPolicy.evaluate()`, `SafetyRule.check()`, `VerifiedCommand.step()`	Decorator, Strategy	1:24
F10	UC-10	`MissionReport`, `LlmDebriefer.write()`, `DebriefRules.advicePossible()`	Template Method, Strategy, Decorator	1:38, 3:49
N01	UC-11	`MissionControl.abort()`, `CommandQueue.cancelWhere()`	Command, Facade	1:38

A.2 How to Build and Run

Prerequisites are Java 21 or newer and Maven 3.9 or newer. The key is entered locally; no key is stored in the repository.

1. Copy the example configuration: `cp .env.example .env`
2. Edit `.env`. The default `LLM_PROVIDER=mock` needs no key and no network. For Gemini set `LLM_PROVIDER=gemini` and put your key in `LLM_API_KEY`.
3. Build: `mvn -q compile`
4. Start the GUI: `mvn -q exec:java`
5. Run a CLI command, for example: `mvn -q exec:java -Dexec.mainClass=com.fleetops.cli.FleetOpsCli -Dexec.args="scan"`

Command	What it does

A.3 Command-Line Reference

Command	What it does
`config`	Shows the effective configuration. Keys are never printed.
`scan [-t flood,fire] [-v]`	Reads one new item per topic, has the AI verify it and stores new incidents.
`incidents [-a] [-s STATE] [-t TYPE]`	Lists incidents; active ones unless `-a` or `-s` is given.
`show <id>`	Shows one incident.
`run <id>`	Plans the response, asks for approval, runs the simulation and prints the report and the debrief. Options: `--dry-run`, `-y`, `--progress`, `--abort-after SECONDS`, `--no-debrief`, `--send-drones N`, `--send-helicopters N`, `--send-robodogs N`, `--pattern NAME`, `--rovers`, `--drones`, `--robodogs`, `--helicopters`, `--depot-km`, `--seed`, `--max-ticks`.
`reset [-f] [-y]`	Deletes stored incidents after confirmation; `-f` removes only finished ones.

Exit codes: 0 success, 1 configuration error, 2 bad usage, 3 incident not found, 4 failed or not completed.

A.4 Configuration Keys

Key	Meaning	Default
`LLM_PROVIDER`	`mock`, `gemini` or `ollama`	`mock`
`LLM_API_KEY`	API key, required for `gemini`	none
`LLM_MODEL`	Model name	provider default (`gemini-2.5-flash`)
`LLM_FALLBACK_MODELS`	Gemini models to try, in order, when the main one has used its daily allowance	none
`LLM_BASE_URL`	Ollama server address	`http://localhost:11434`
`DB_URL`	JDBC address of the incident database, or `memory`	`jdbc:h2:./data/fleetops`
`NEWS_SOURCES`	`demo`, `google`, `telegram`, comma separated, read in the order listed	`demo`
`TELEGRAM_CHANNELS`	Public channel names for the `telegram` source	none
`NEWS_GOOGLE_EDITION`	Google News language and country	`hl=en-CA&gl=CA&ceid=CA:en`
`NEWS_TOPICS`	`flood`, `famine`, `fire`, `earthquake`	all four
`MAP_TILES`	`dark` (CARTO, needs a key) or `osm`	`dark`
`CARTO_API_KEY`	Free CARTO basemap key	none

A.5 Package Overview

Package	**Role**
`domain`	Incidents, survivors, geography, the fleet (rover, drones, robodogs, helicopters)
`sim`	Fixed-step simulation engine, world, survivor placement
`command`, `strategy`	Mission commands with a per-unit queue; search patterns
`safety`	Safety rules, the policy and the verified-command decorator
`news`, `ai`, `intel`	News sources and article text, LLM clients, the verify-and-store pipeline
`plan`, `mission`	Planners, the compiler, `MissionControl` and the rescue dispatcher
`debrief`	AI debrief with a rule-based fallback
`repository`	In-memory and JDBC incident storage
`app`, `config`	Configuration and the composition root
`cli`, `gui`	The two user interfaces